

Section 1: Define Your Agent

1.1 Project Description

What problem does the project solve?

AI Smart Recipe and Meal Planning will be a GUI designated to help users manage and organize their grocery, meals, recipe, and track their health. It can also be used through command-line interface(CLI).

Who are the target users?

The target users are individuals who want assistance with organizing their meals and groceries, and looking for recipes. The project is also meant to be user friendly for people with specific dietary constraints, and individuals who want to track their nutrition intake.

What can the agent do?

The AI agent will provide assistance in generating recipes and groceries based on their configuration. In addition, it will be used to source information from a database to track their nutritional intake, and assist with their meal planning.

Why is an AI agent appropriate for this problem?

An AI agent is appropriate for this problem because it requires making decisions based on changing user constraints and looking for information that satisfies the user's request. It is also appropriate as this project is under time constraint, and it would be difficult to complete with a pure deterministic system.

Which AI/LLM models are used?

This project will be done with the help of Claude. I am planning to use *Sonnet 5* or *Opus 5.5*.

How does the AI model interact with the rest of the software system?

The agent that is part of the program talks to the AI model. When the user asks for something, the agent sends Claude the request and related information, such as the user's profile and pantry. Claude will respond with an answer or ask the program for one of its tools, such as searching recipes or nutritional summary. Once it has the necessary solution, it will be given back to the program for further approval.

1.2 Feature Specification

F01 — Weakly Meal Calendar (Hybrid)

The user will be able to access a calendar with 'meal slots' and 'snack slots'. Within the slots, users will be able to add a 'meal' and set up a serving/quantity of Ingredients Meals whose ingredients in the pantry aren't enough will be flagged. Accessed through the GUI, a separate menu.

F02 — Shopping Checklist (Deterministic)

The Shopping Checklist would allow the user to list grocery items and quantity, each item can be ticked off. Upon pressing 'Finish Shopping', ticked items will be moved onto the pantry, and the checklist will reset. There could be multiple checklists at a time. If such ingredients already exist in the pantry it would be flagged. Accessed through the GUI, a separate menu.

F03 — Shopping List Generation (Hybrid)

The user would be able to select a 'meal' from their recipe library to generate a shopping list. This list when approved by the user will be added to another checklist or separate checklist. The agent will use the meal library and other tools to form the checklist, accessed through the GUI, the same menu as the shopping checklist.

F04 — Pantry Inventory (Deterministic)

The pantry tab would have a list of available ingredients. The user will be able to edit it manually, being able to change 'quantity', or 'add', and 'remove' ingredients. Invalid quantities or units are rejected with a message, and unknown ingredient names are rejected with a message, Accessed through the GUI, separate menu.

F05 — Macro Generator (Hybrid)

The user will be able to select a 'meal(s)' from the recipe library, or 'ingredients' from the pantry to get a nutrition summary, they would be able to select the serving or amount. This will include calories, carbohydrates, protein, fat, sugar, vitamins. The agent will use a reliable 'source' to form a calculation. Invalid quantities, units, or ingredients, are rejected with a message. Accessed through the GUI, same menu as 'recipe library', 'Meal Calendar'.

*May used USDA or Canadian Nutrient File as source

F06 — Profile Configuration (Deterministic)

In the Profile tab, the user is able to set 'dietary restriction', 'allergy', favourite 'cuisines'. This will be an important reference for the AI agent, with 'dietary restriction' and 'allergy' ranked higher in reference. Accessed through the GUI, a separate menu. Unknown allergy names are rejected with a message.

F07 — Recipe Recommendations (Hybrid)

The user will be asked to set up a preference filter: 'cuisine', 'cooking time', 'nutritional goals',

‘dietary restriction/allergy’. The ai agent will then look for a recipe that fits these requirements. The user will be able to approve if they want it added in the recipe library. If it is unable to find any, it will ask the user to ‘loosen’ filters. Accessed through the GUI, a separate menu.

F08 — Recipe Library (Deterministic)

The user will be able to ‘edit’, ‘add’ or ‘remove’ recipes. There will be a slot for ‘ingredients’ and ‘cooking instructions’, and ‘dietary instructions/allergic warning’, then a ‘nutritional summary’ (uses the macro generator). Each Ingredient must have a unit/serving or it will be flagged. Accessed through the GUI, a separate menu.

F09 — Favourite Recipe (Deterministic)

The user clicks the heart icon on a recipe to add or remove it from favourites. The program saves the choice and shows the recipe under the Favourites filter. Favourites are also ranked higher in recommendation. Accessed through the GUI, same menu as ‘recipe library’.

F10 — Nutrition Summary (Hybrid)

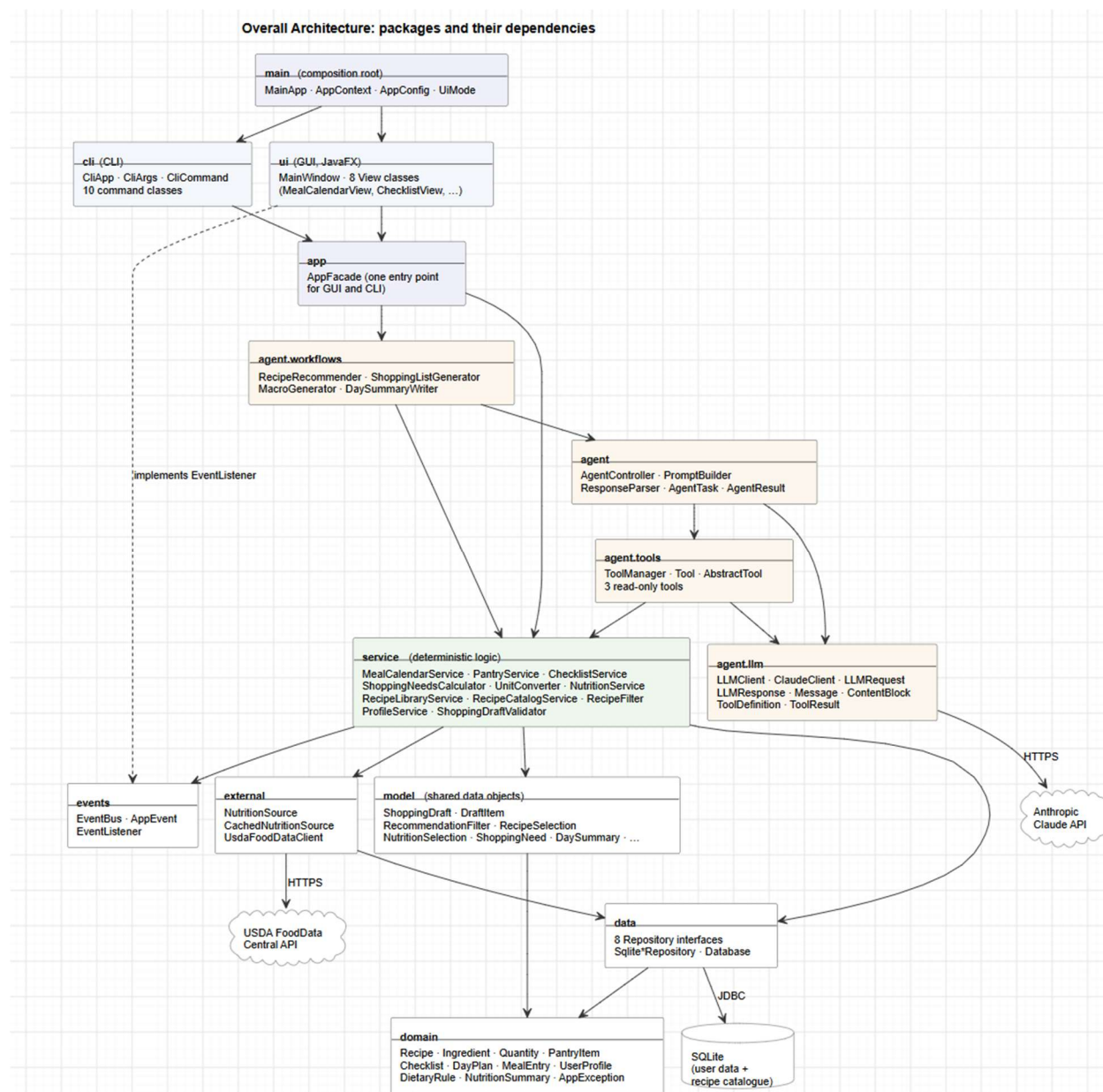
For each day in the calendar, the user will be able to open a “Nutritional Summary” of the meals planned that day. The total will be compared with the user’s daily nutritional goal profile. The user will then receive a summary of the day via AI, reviewing how well the user have done. The user can see the totals for the whole week. If nutrition data is missing, the summary is marked incomplete. Invalid goals are rejected with a message. Access through the GUI, same menu as ‘Meal Calendar’.

*May use USDA or Canadian Nutrient File as source

Section 2: UML Design

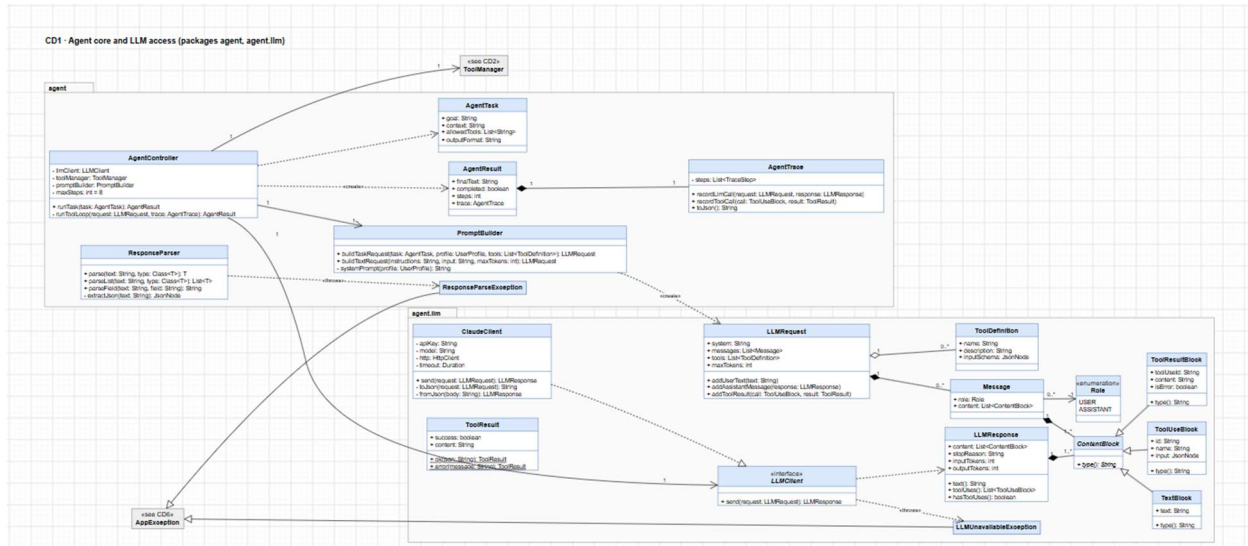
LINK: [drawiolink](#)

2.1: Class Diagram



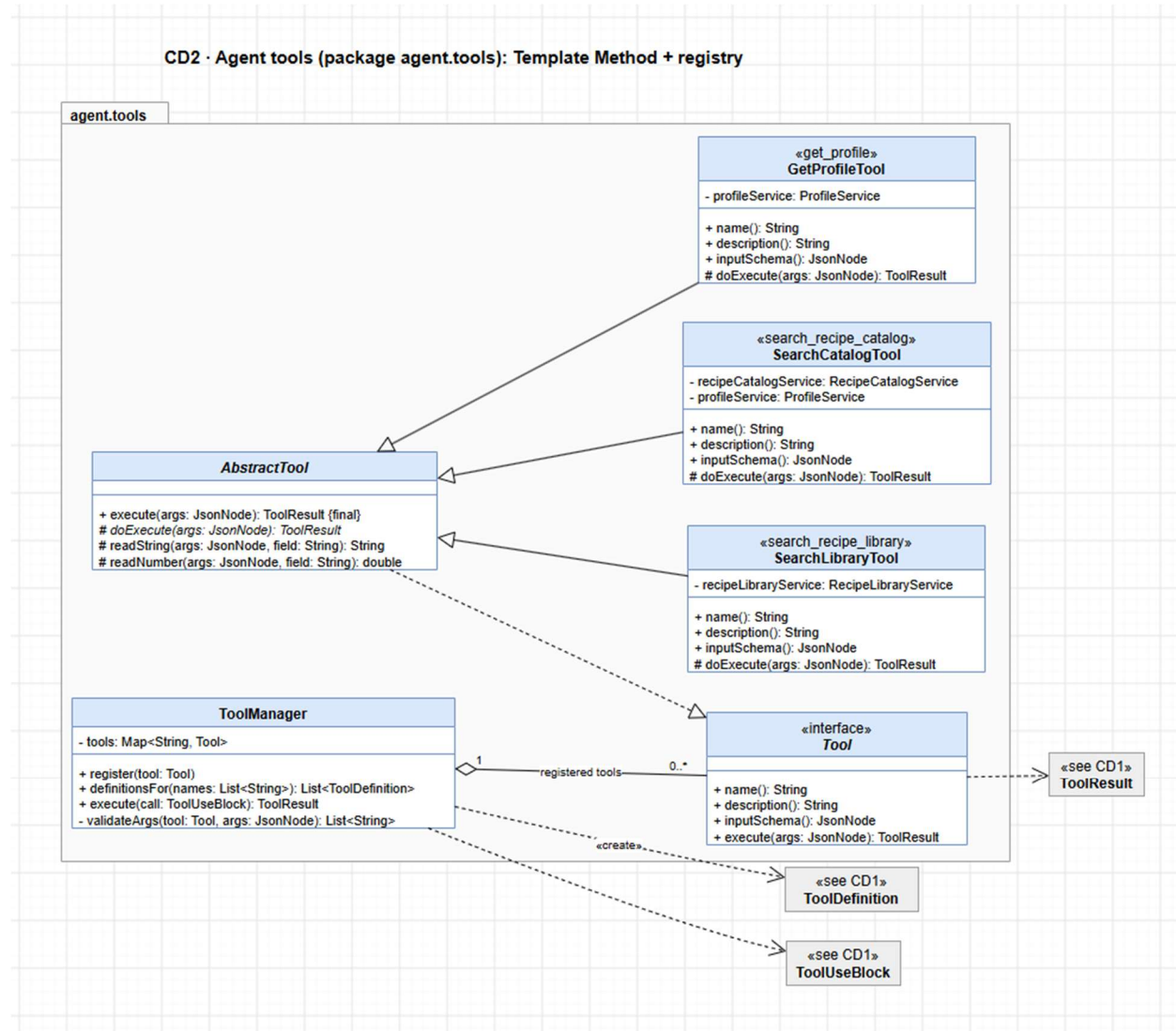
CD1: Agent core and LLM access

AgentController runs the agent's tool-use loop: it sends Claude an AgentTask with the tools the task may use, executes the tools Claude asks for through ToolManager, and returns an AgentResult (at most 8 steps). **PromptBuilder** and **ResponseParser** are generic: they build requests from text the workflows supply and turn Claude's JSON into any model object (parse(text, type)), so the core knows nothing about individual features. Messages follow the Anthropic Messages API: a Message holds a list of **ContentBlocks** (TextBlock, ToolUseBlock, ToolResultBlock), and each tool answer goes back as a ToolResultBlock that carries the id of the ToolUseBlock it answers. All model access goes through the **LLMClient** interface; **ClaudeClient** is the adapter for the Anthropic API.



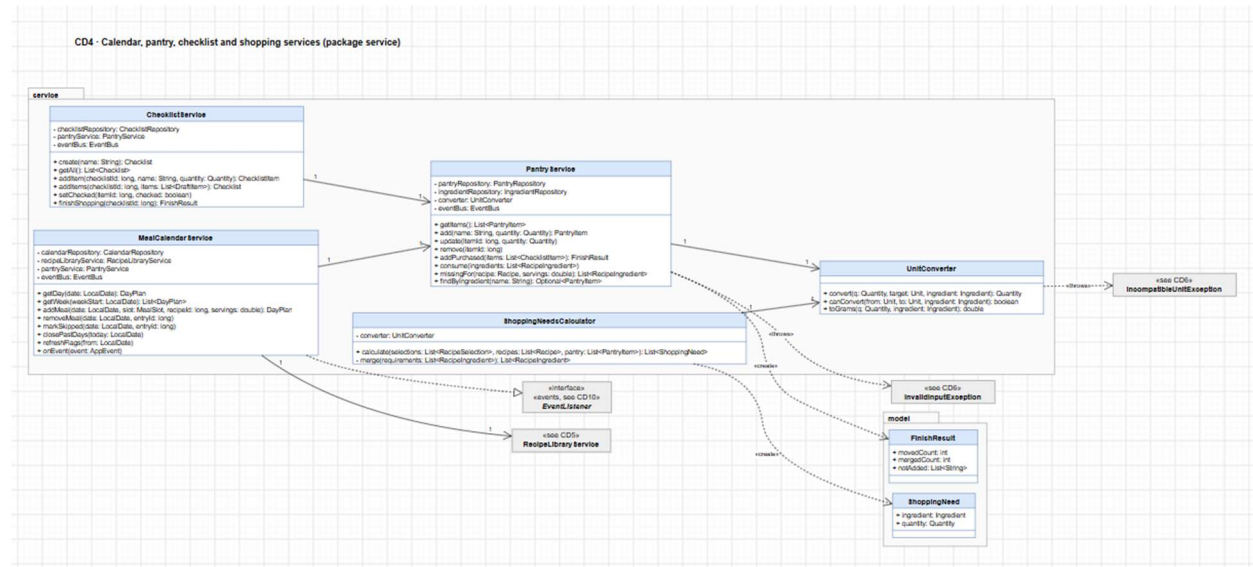
CD2: Agent tools

ToolManager keeps the registered tools, gives Claude their definitions and checks each call against the tool's schema before running it. AbstractTool.execute() is a final template method; each tool only implements doExecute(). The three tools (get_profile, search_recipe_library, search_recipe_catalog) only read data, so Claude can never change the user's data directly.



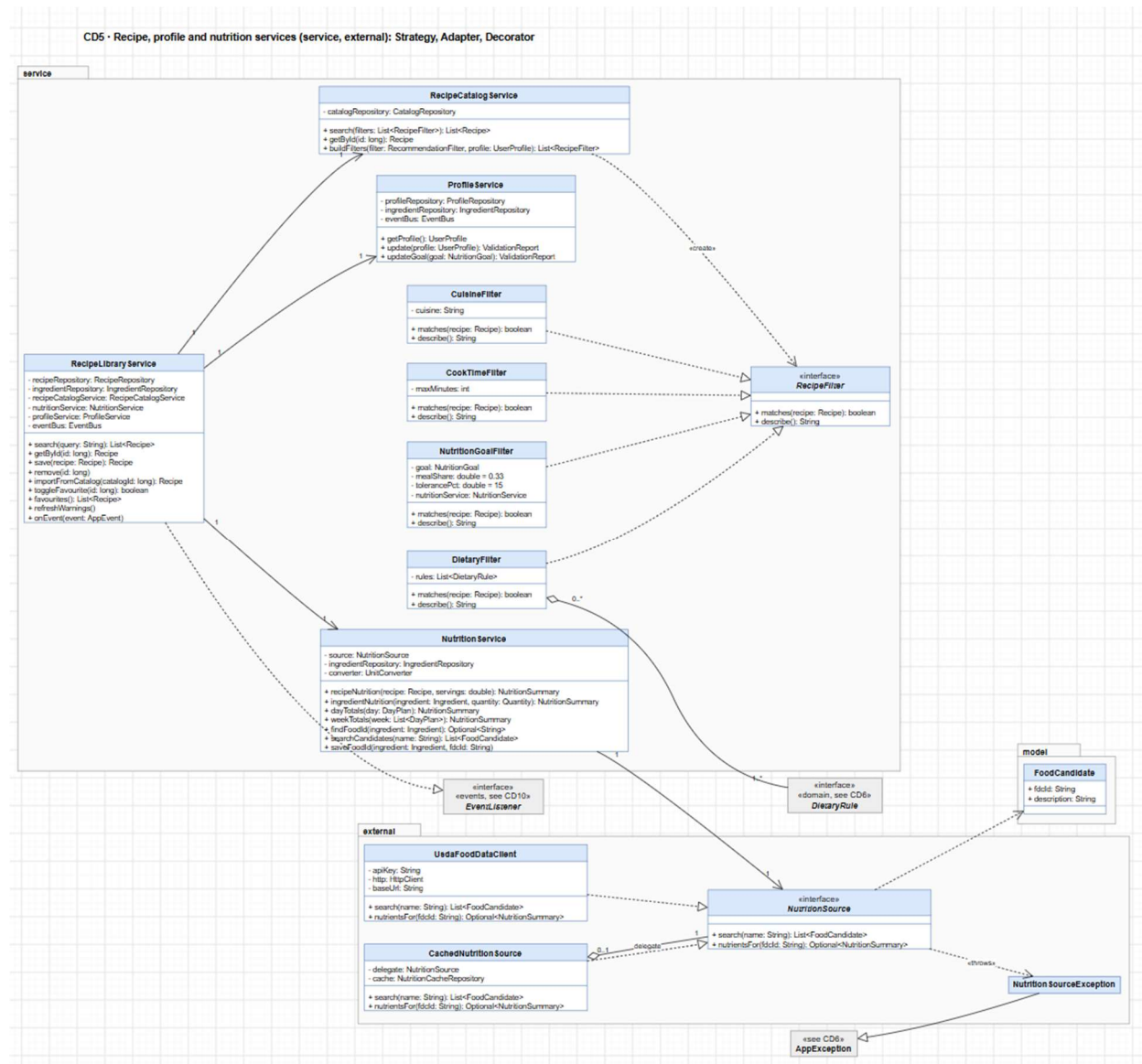
CD4: Calendar, Pantry, checklist and shopping services

MealCalendarService manages the weekly calendar (F01): it returns a week of days, adds meals, flags meals the pantry cannot cover, and removes used ingredients at the end of the day. PantryService (F04) validates and merges pantry items, ChecklistService (F02) manages several checklists, and ShoppingNeedsCalculator (F03) does the exact quantity arithmetic with UnitConverter.



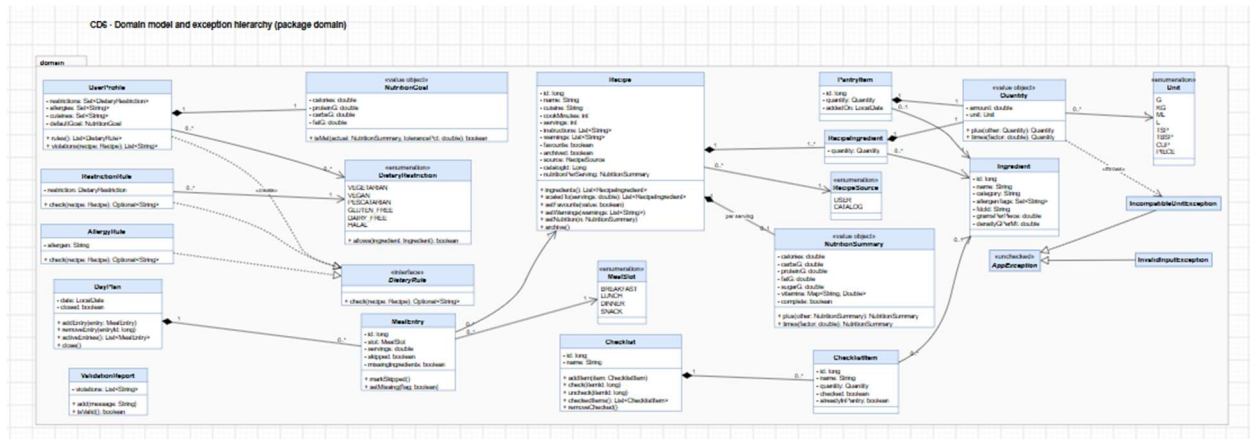
CD5: Recipe, profile and nutrition services

RecipeLibraryService handles recipes and favourites (F08, F09) and archives removed recipes. RecipeCatalogService searches the built-in catalogue with RecipeFilter strategies (F07). ProfileService saves the profile and the daily nutrition goals (F06, F10). NutritionService calculates nutrients for recipes, days and weeks (F05, F10) through the NutritionSource interface: CachedNutritionSource (Decorator) wraps UsdaFoodDataClient (Adapter).



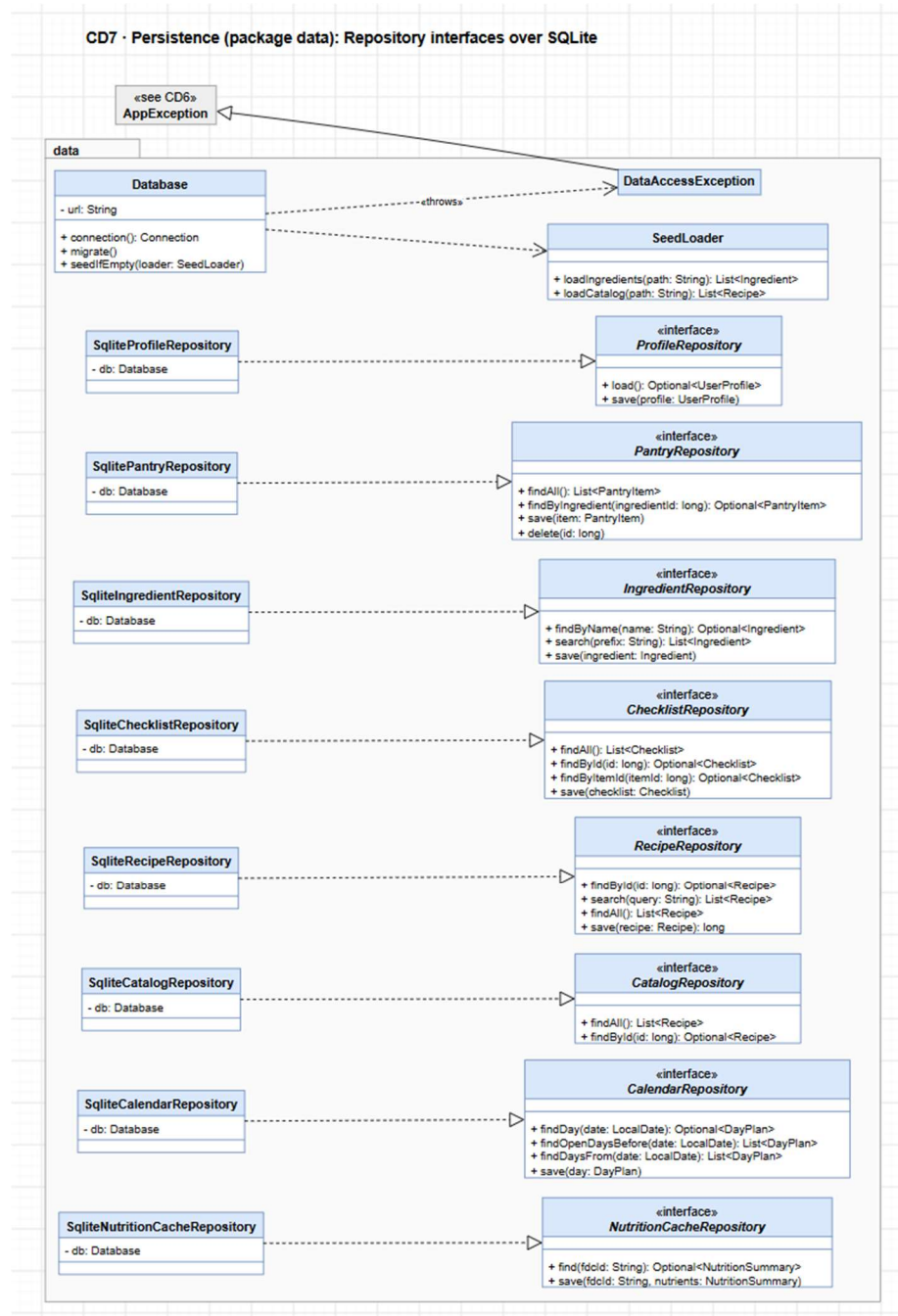
CD6: Domain model and exceptions

A Recipe is made of RecipeIngredients, a DayPlan holds the MealEntry objects of one day, and a Checklist holds ChecklistItems. UserProfile holds the dietary settings and the daily NutritionGoal, and creates the DietaryRule objects. Quantity, NutritionGoal and NutritionSummary are value objects. Every exception is an unchecked subclass of ApplicationException.



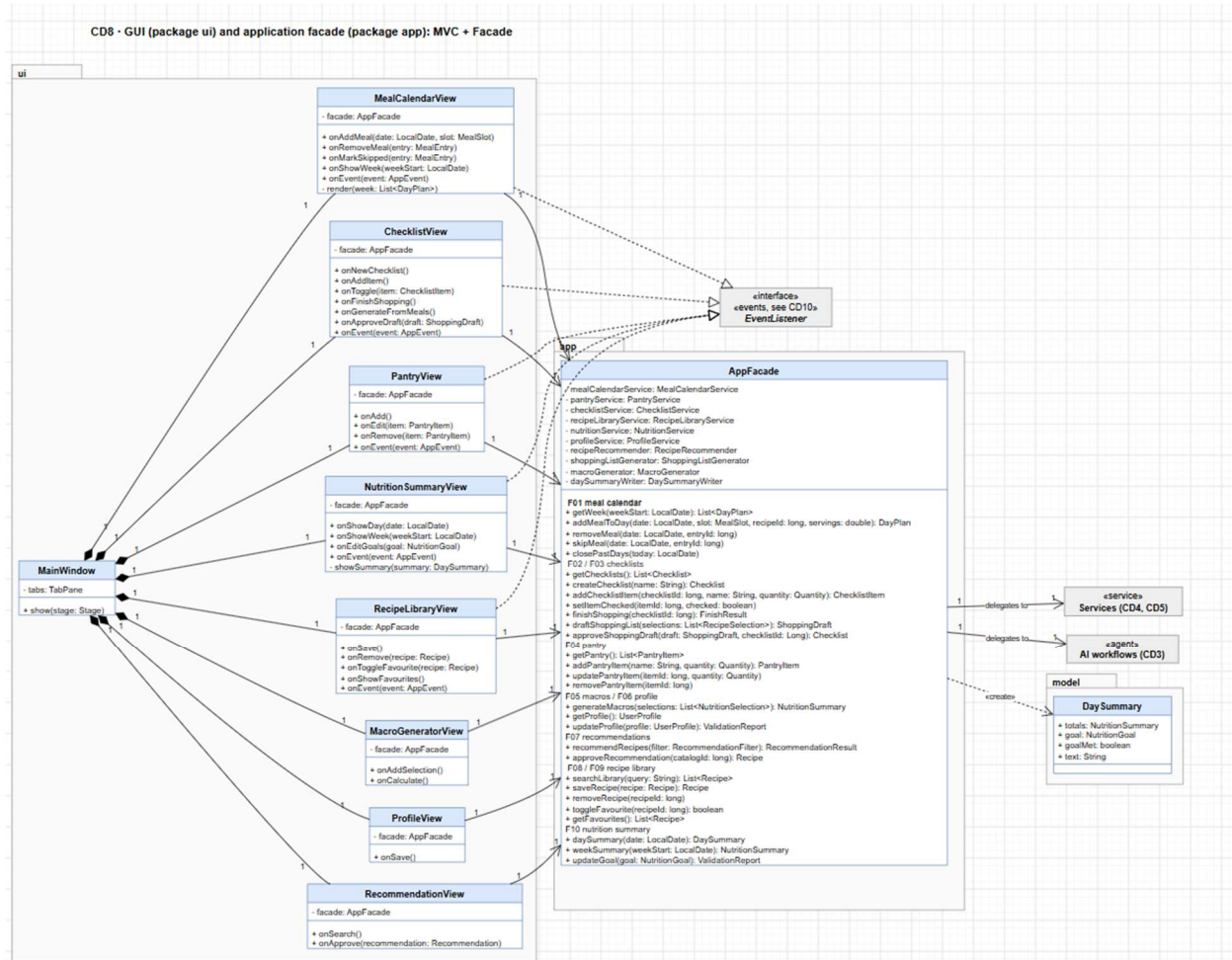
CD7: Persistence

Repository interfaces keep SQL out of the services. Each one has a SQLite implementation, and they all share one Database object. The recipe catalogue and the ingredient list are loaded from JSON files.



CD8: GUI and application façade

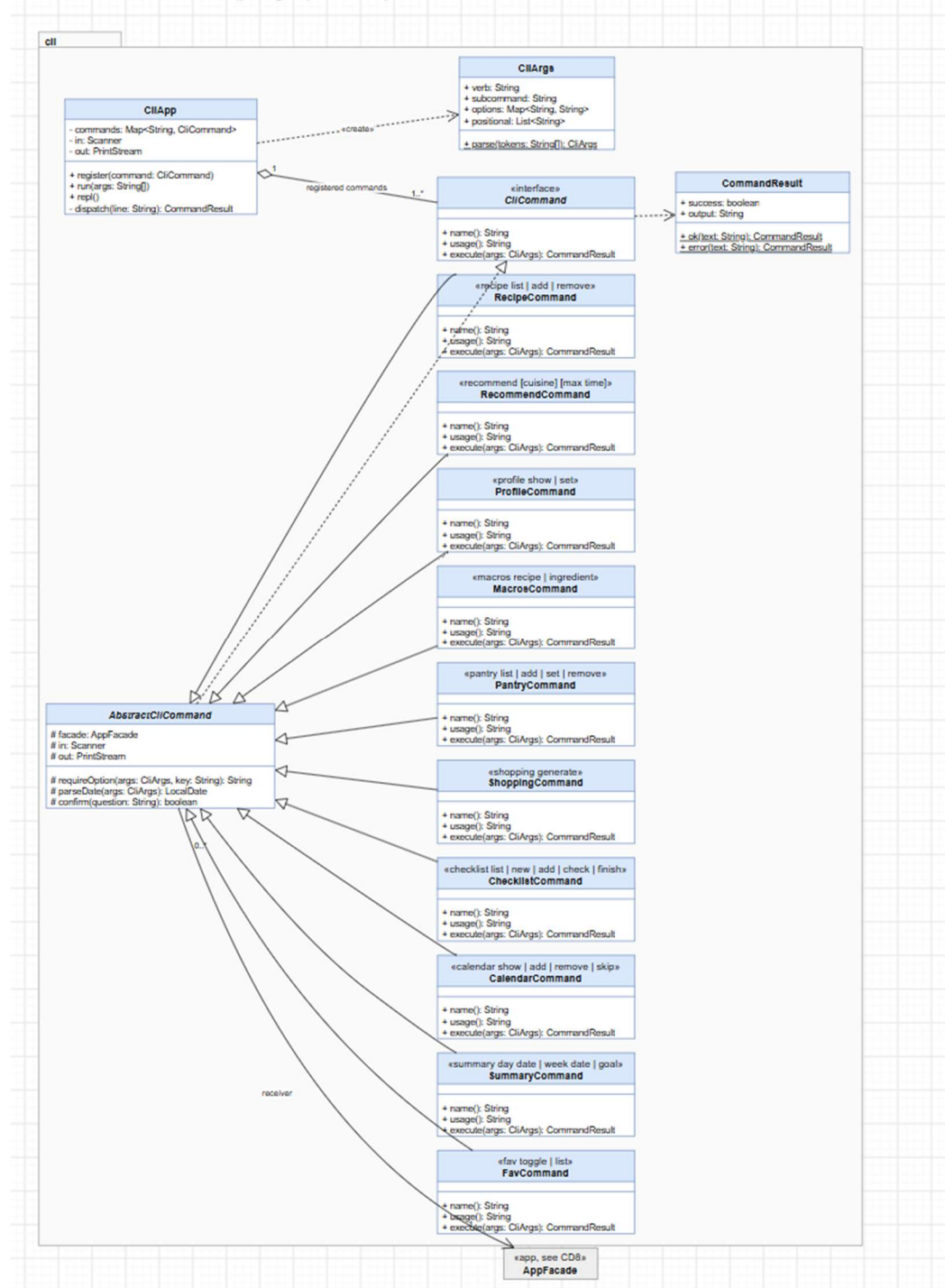
MainWindow has one view per menu (eight views, including NutritionSummaryView for F10). Each *View class is the JavaFX controller of its .fxml layout (MVC). Every view and CLI command calls AppFacade, whose methods are grouped by feature (Facade). Views that must refresh implement EventListener, and AI calls run in a background task.



CD9: Command-line interface

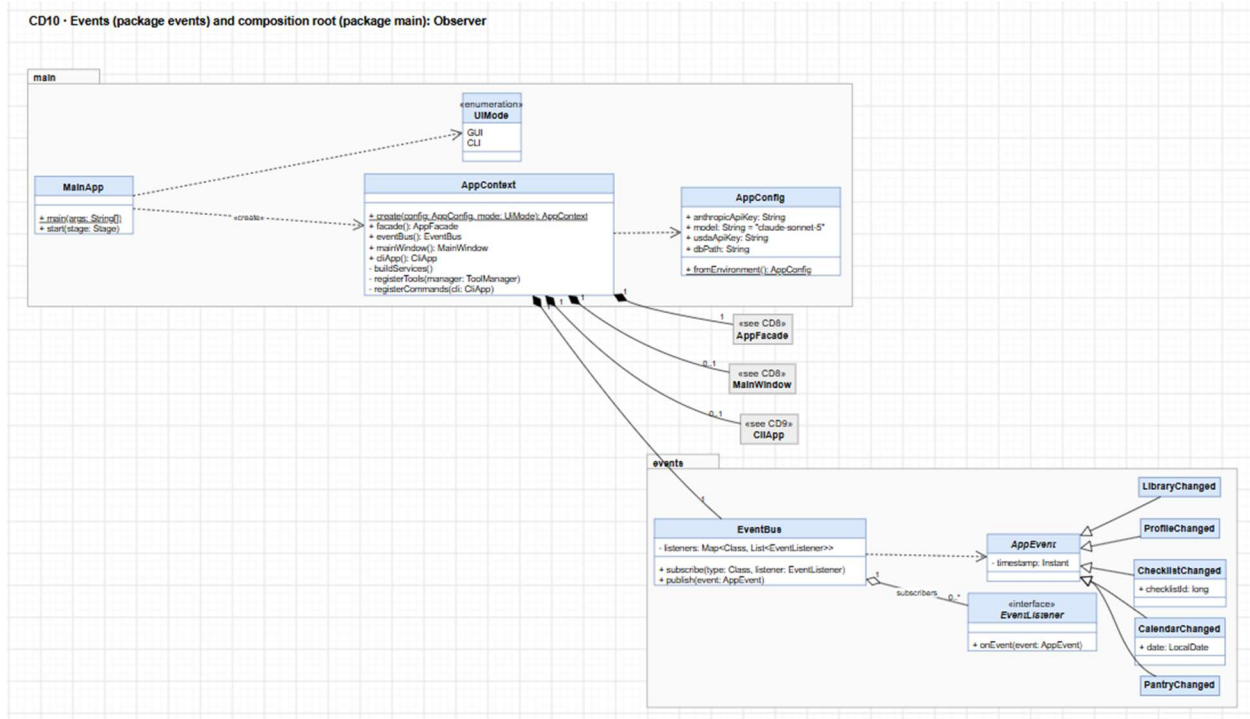
CliApp holds the registered commands. Each command is a Command object: AbstractCliCommand holds the receiver (AppFacade) and the console, and the ten concrete commands (one per feature, for example summary day 2026-10-07) implement name(), usage() and execute().

CD9 - Command-line interface (package cli): Command pattern



CD10: Events and composition root

EventBus keeps the listeners and publishes AppEvent subclasses (Observer). MainApp starts the program in GUI or CLI mode, and AppContext reads the settings (API keys, model name, database path), builds every object, and registers the tools and commands. Nothing depends on the main package, so there are no package cycles.



Design Pattern

Pattern 1: Facade

What design problem does the pattern address?

The GUI (seven views) and the CLI (ten commands) both need all ten features, which are spread over many services and five AI workflows. Without one entry point, every view and command would have to know which classes to call and in what order.

Which classes participate, and what role does each play?

Class(es)	Role
AppFacade	Facade: one simple method per user action (addMealToDay, finishShopping, planDay, ...).
MealCalendarView, ChecklistView, RecipeLibraryView; CalendarCommand, PantryCommand,	Clients: call only the facade.
MealCalendarService, PantryService, ChecklistService, RecipeLibraryService, NutritionService, MealPlanner, RecipeRecommender, ...	Subsystem: does the actual work.

Why is this pattern appropriate?

Both interfaces stay thin and behave the same because they call the same facade method. The subsystem can change without touching the GUI or CLI. The facade is large but grouped by feature, which we accept to keep a single entry point.

What would be more difficult without it?

The same calling logic would be copied into 7 views and 10 commands, the GUI and CLI could behave differently, and every UI class would depend on many services.

Pattern 2: Command

What design problem does the pattern address?

The CLI must map many text commands with sub-commands and options to application operations, with the same output and error handling for all of them.

Which classes participate, and what role does each play?

Class(es)	Role
CliCommand	Command interface: name(), usage(), execute(args).
AbstractCliCommand	Shared base: holds the receiver and the console in/out, plus helpers (parseDate, confirm).
CalendarCommand, ChecklistCommand, ShoppingCommand, PantryCommand, MacrosCommand, ProfileCommand, RecommendCommand, RecipeCommand, FavCommand, SummaryCommand	Concrete commands: turn arguments into one facade call.
CliApp	Invoker: finds the command by name and runs it.
AppFacade	Receiver: performs the operation.

Why is this pattern appropriate?

Every command has the same execute() signature, so CliApp dispatches through a map, help text comes from usage(), and each command can be tested on its own.

What would be more difficult without it?

One huge switch statement in main() would mix parsing and business calls, adding a command could break the others.

Pattern 3: Observer

What design problem does the pattern address?

One change affects several parts of the app: adding pantry items can remove the “not enough ingredients” flag on calendar meals (F01), finishing shopping changes the pantry (F02/F04), and changing an allergy changes recipe warnings (F06/F08).

Which classes participate, and what role does each play?

Class(es)	Role
EventBus	Subject: keeps subscribers and publishes events.
EventListener	Observer interface: onEvent(event).
PantryChanged, CalendarChanged, ChecklistChanged, ProfileChanged, LibraryChanged	Events.
PantryService, MealCalendarService, ChecklistService, ProfileService, RecipeLibraryService	Publishers.
MealCalendarService, RecipeLibraryService, MealCalendarView, ChecklistView, PantryView, RecipeLibraryView, NutritionSummaryView	Observers: update flags, warnings or the screen.

Why is this pattern appropriate?

Publishers do not need to know who depends on them

What would be more difficult without it?

PantryService would need references to the calendar and every view; forgetting one call would leave stale flags on the screen.

Pattern 4: Adapter

What design problem does the pattern address?

The Anthropic API and the USDA FoodData Central API use their own JSON formats, authentication and error codes, which do not match the app's own types.

Which classes participate, and what role does each play?

Class(es)	Role
LLMClient, NutritionSource	Targets: interfaces the app uses.
ClaudeClient, UsdaFoodDataClient	Adapters: translate requests and replies (e.g. content blocks to and from API JSON); turn network errors into LLMUnavailableException / NutritionSourceException, declared on the interfaces.
Anthropic Claude API, USDA FoodData Central API	Adaptees: the external services.
AgentController, workflows; NutritionService, CachedNutritionSource	Clients.

Why is this pattern appropriate?

The rest of the app never sees vendor JSON, and tests can use a fake client with scripted answers instead of the real API.

What would be more difficult without it?

API details would be spread across many classes; changing the model or testing without internet would be hard.

Pattern 5: Decorator

What design problem does the pattern address?

Nutrition look-ups are needed often (macro generator, recipe library, daily totals, meal planning), but the USDA API is slow, rate-limited and needs the internet.

Which classes participate, and what role does each play?

Class(es)	Role
NutritionSource	Component interface: search(name), nutrientsFor(fdclid).
UsdaFoodDataClient	Concrete component: calls the USDA API.
CachedNutritionSource	Decorator: same interface and same contract; returns cached values or asks the wrapped source, stores the result and passes exceptions through unchanged.
NutritionService	Client: does not know whether a value came from the cache.

Why is this pattern appropriate?

Caching is added without changing the USDA adapter or NutritionService.

What would be more difficult without it?

Caching code would be mixed into NutritionService or the adapter, and every repeated look-up would call the API.

2.2: Use-Case Diagram

Who are the actors?

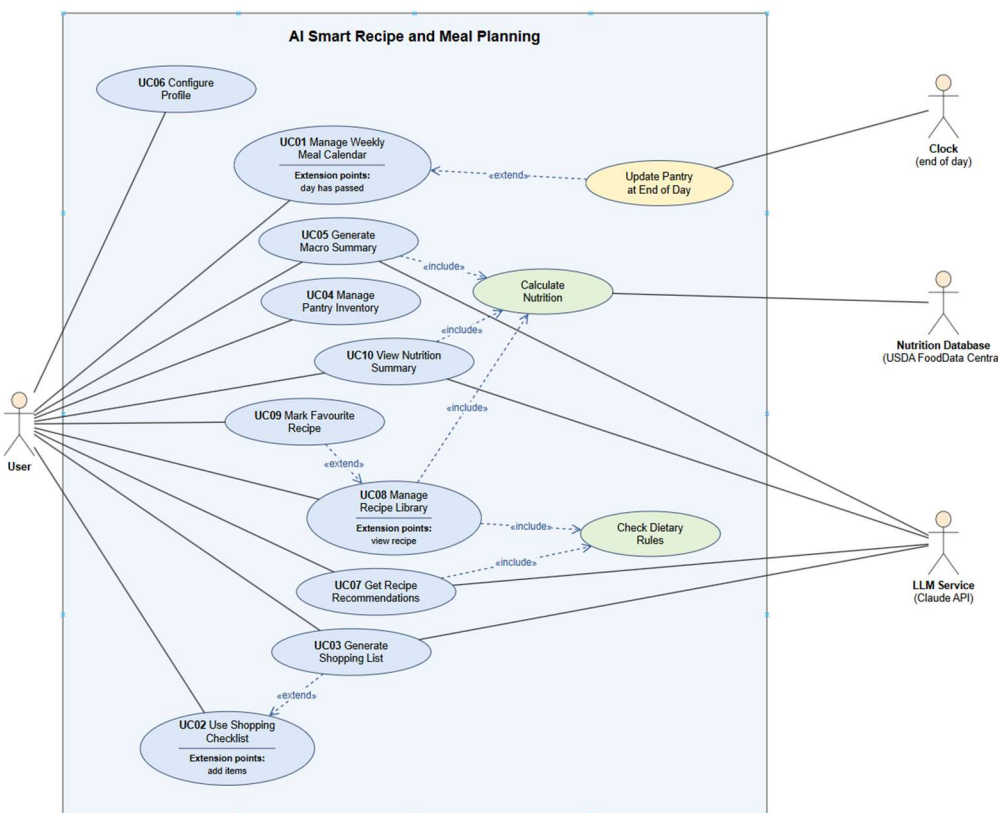
- **User (primary actor):** A user managing/organizing their meals or groceries. They use the GUI or the CLI.
- **LLM Service — Claude API (AI service):** The agent talks to Claude in the four Hybrid use cases: UC03, UC05, UC07 and UC10.
- **Nutrition Database — USDA FoodData Central (external API):** Gives the nutrient values used by Calculate Nutrition.
- **Clock (timer):** Starts the end-of-day pantry update when a day has passed.
- **Administrator:** Not applicable. The user manages their own profile and data.

The agent's tools (such as searching recipes and profile) are part of the system, so they are not actors.

How do the relationships in the diagram work?

- Solid lines show which actors take part in a use case.
- «include» marks a step that several use cases share. Calculate Nutrition is included by UC05, UC08 and UC10. Check Dietary Rules is included by UC07, and UC08.
- «extend» marks optional behaviour added at an extension point, which is written inside the base use case. Update Pantry at End of Day extends UC01 at “day has passed”, UC03 extends UC02 at “add items”, and UC09 extends UC08 at “view recipe”.

Diagram:



UC01: Manage Weekly Meal Calendar

Use Case ID: UC01

Use Case Name: Manage Weekly Meal Calendar

Actor(s): User (primary), Clock (through Update Pantry at End of Day)

Goal: Plan meals and snacks for each day of the week and keep the pantry up to date.

Preconditions: The recipe library has at least one recipe.

Trigger: The user opens the Meal Calendar menu.

Main Success Scenario:

1. The system shows the current week, with meal and snack slots for each day.
2. The user picks a day and a slot (moving to another week if needed).
3. The user chooses a recipe from the library and sets the servings.
4. The system checks the pantry. If there are not enough ingredients, the meal is flagged.
5. The system saves the meal and shows it in the slot.

Alternative/Exception Flows:

- 2a** The user removes a meal, or marks it as skipped.
- 3a** Invalid servings: the system shows an error and nothing is saved.
- *a** At any time, if the pantry changes, the system re-checks the flags on upcoming meals.
- Ext.** Extension “day has passed” (Update Pantry at End of Day): when the app starts or at midnight, the Clock triggers the update. The system removes the ingredients of that day's meals from the pantry.

Postconditions: The week is saved with up-to-date flags. Past days are closed and the pantry is updated.

Related Feature(s): F01 Weekly Meal Calendar (also F04 Pantry Inventory).

UC02: Use Shopping Checklist

Use Case ID: UC02

Use Case Name: Use Shopping Checklist

Actor(s): User (primary)

Goal: Keep grocery checklists, tick items off while shopping, and move bought items into the pantry.

Preconditions: None.

Trigger: The user opens the Shopping menu.

Main Success Scenario:

1. The user opens a checklist or creates a new one.
2. The user adds items with a quantity and unit (extension point “add items”).
3. The system checks the pantry and flags items that are already there.
4. While shopping, the user ticks items off.
5. The user clicks Finish Shopping.
6. The system moves the ticked items into the pantry and adds them to items that are already there.

Alternative/Exception Flows:

- 3a** The item is already in the pantry: it is still added, with a flag such as “already in pantry: 6”.
- 6a** Unknown ingredient name: the item is not moved and is listed as “not added”.
- 6b** The units cannot be combined (for example grams and pieces): the item becomes a separate pantry entry.

Postconditions: The pantry includes the bought items. The checklist keeps only the unticked items.

Related Feature(s): F02 Shopping Checklist (also F04 Pantry Inventory).

UC03: Generate Shopping List

Use Case ID: UC03

Use Case Name: Generate Shopping List

Actor(s): User (primary), LLM Service

Goal: Create shopping items from chosen recipes, minus what is already in the pantry, and add them to a checklist.

Preconditions: The chosen recipes are in the recipe library.

Trigger: In the Shopping menu, the user clicks “Generate from meals” while adding items (extends UC02 at “add items”).

Main Success Scenario:

1. The user selects one or more recipes and the number of servings.
2. The system adds up the ingredients and subtracts what is already in the pantry.
3. The agent sends the list to Claude, which merges duplicates.
4. The system shows the draft list.
5. The user approves it and chooses an existing checklist or a new one to add it into.

Alternative/Exception Flows:

- 2a** The pantry already has everything: the system shows “Everything is already in your pantry”.
- 3a** Claude is unavailable, or its answer fails the check in step 4: the system shows the plain list from step 2.
- 5a** The user rejects the draft: nothing is added.

Postconditions: The approved items are on a checklist.

Related Feature(s): F03 Shopping List Generation (also F02, F04, F08).

UC04: Manage Pantry Inventory

Use Case ID: UC04

Use Case Name: Manage Pantry Inventory

Actor(s): User (primary)

Goal: Keep the list of ingredients at home correct.

Preconditions: The list of known ingredients is loaded.

Trigger: The user opens the Pantry menu.

Main Success Scenario:

1. The user adds an ingredient, changes its quantity, or removes it.
2. The system checks that the ingredient name is known.
3. The system checks the quantity and unit.
4. The system saves the change. If the ingredient is already in the pantry, the quantities are added together.
5. The system re-checks the flags on upcoming calendar meals and refreshes the pantry list.

Alternative/Exception Flows:

- 2a** Unknown ingredient name: the change is rejected with a message.
- 3a** Invalid quantity or unit: the change is rejected with a message.

Postconditions: The pantry is saved and the calendar flags are up to date.

Related Feature(s): F04 Pantry Inventory (also F01 meal flags).

UC05: Generate Macro Summary

Use Case ID: UC05

Use Case Name: Generate Macro Summary

Actor(s): User (primary), LLM Service, Nutrition Database

Goal: See calories, carbohydrates, protein, fat, sugar and vitamins for chosen meals or ingredients.

Preconditions: None. Nutrition data is fetched when it is needed.

Trigger: The user opens the Macro Generator from the Recipe Library or Meal Calendar menu.

Main Success Scenario:

1. The user selects recipes and/or pantry ingredients and sets the servings or amounts.
2. The user clicks Calculate.
3. For each ingredient that is not yet matched to a USDA food, the system gets a short list of possible matches and the agent asks Claude to pick the right one.
4. The system gets the nutrient values from the USDA database and scales them to the amounts (includes Calculate Nutrition).
5. The system adds everything up and shows the summary.

Alternative/Exception Flows:

- 1a Invalid amount, unit or ingredient: rejected with a message.
- 3a Claude is unavailable: the system uses the top USDA match and marks it “check match”.
- 4a A food is not found, or the USDA database cannot be reached: the summary is marked “incomplete”.

Postconditions: The summary is shown. Food matches and nutrient values are saved for next time.

Related Feature(s): F05 Macro Generator.

UC06: Configure Profile

Use Case ID: UC06

Use Case Name: Configure Profile

Actor(s): User (primary)

Goal: Set dietary restrictions, allergies and favourite cuisines for the AI features to follow.

Preconditions: None.

Trigger: The user opens the Profile menu.

Main Success Scenario:

1. The user selects dietary restrictions, enters allergies and picks favourite cuisines.
2. The user clicks Save.
3. The system checks each value. Allergy names must be known ingredients.
4. The system saves the profile.
5. The system re-checks every recipe in the library and updates its allergy and diet warnings.

Alternative/Exception Flows:

- 3a Unknown allergy name or empty value: the system suggests close names, or saves nothing.

Postconditions: The profile is saved. Restrictions and allergies are strict rules in UC07, and UC08, favourite cuisines are only preferences.

Related Feature(s): F06 Profile Configuration (also F08 warnings)

UC07: Get Recipe Recommendations

Use Case ID: UC07

Use Case Name: Get Recipe Recommendations

Actor(s): User (primary), LLM Service

Goal: Find recipes that match the user's filters and add the ones they like to the library.

Preconditions: The built-in recipe catalogue is loaded.

Trigger: The user opens the Recommendations menu.

Main Success Scenario:

1. The user sets the filters: cuisine, cooking time and nutritional goal. Dietary restrictions and allergies come from the profile.
2. The user clicks Search.
3. The agent sends the request to Claude together with the user's favourite recipes.
4. Claude asks the system to search the recipe catalogue. The system returns only recipes that pass the filters.
5. The system removes any pick that breaks a dietary rule (includes Check Dietary Rules) and shows the rest as cards.
6. The user approves a recipe and the system adds it to the recipe library.

Alternative/Exception Flows:

- 4a** Claude is unavailable: the system shows the first recipes that pass the filters, without explanations.
- 5a** No recipes are left: the system shows "No recipes found. Try loosening your filters."
- 6a** The user approves nothing: the library does not change.

Postconditions: Approved recipes are in the recipe library.

Related Feature(s): F07 Recipe Recommendations (also F06, F08, F09).

UC08: Manage Recipe Library

Use Case ID: UC08

Use Case Name: Manage Recipe Library

Actor(s): User (primary), Nutrition Database

Goal: Add, edit or remove recipes with ingredients, instructions, warnings and a nutrition summary.

Preconditions: None.

Trigger: The user opens the Recipe Library menu.

Main Success Scenario:

1. The user adds a new recipe or open to edit (change/remove).
2. The user enters the ingredients with amounts and units, and the cooking instructions.
3. The user clicks Save.
4. The system checks that every ingredient is known and has an amount and unit.
5. The system calculates the nutrition per serving (includes Calculate Nutrition).
6. The system adds allergy and diet warnings based on the profile (includes Check Dietary Rules).
7. The system saves the recipe and shows it in the library.

Alternative/Exception Flows:

- 1a** The user removes a recipe: it disappears from the library, but meals already in the calendar keep it.
- 4a** An ingredient is unknown or has no amount or unit: it is flagged and nothing is saved.
- 5a** Nutrition data is missing: the summary is marked "incomplete".

Postconditions: The recipe library is updated.

Related Feature(s): F08 Recipe Library (also F05, F06).

UC09: Mark Favourite Recipe

Use Case ID: UC09

Use Case Name: Mark Favourite Recipe

Actor(s): User (primary)

Goal: Mark recipes as favourites so they are easy to find and preferred by the AI.

Preconditions: The recipe is in the library.

Trigger: While viewing a recipe, the user clicks the heart icon (extends UC08 at “view recipe”).

Main Success Scenario:

1. The user clicks the ‘favourite’ icon
2. recipe appears in (or leaves) the Favourites filter.
3. Favourites are ranked higher in recommendations (UC07)

Alternative/Exception Flows:

- 2a** Saving fails: the heart goes back to how it was and an error is shown.

Postconditions: The favourite status is saved.

Related Feature(s): F09 Favourite Recipe (also F07, F10).

UC10: View Nutrition Summary

Use Case ID: UC10

Use Case Name: View Nutrition Summary

Actor(s): User (primary), LLM Service, Nutrition Database

Goal: See the nutrition of the meals planned for a day or a week, compared with the daily goals.

Preconditions: The calendar has meals for the chosen day or week.

Trigger: In the Meal Calendar menu, the user opens the Nutrition Summary for a day or a week.

Main Success Scenario:

1. The user picks a day (or the whole week).
2. The system calculates the nutrition of every planned meal (includes Calculate Nutrition): calories, carbohydrates, protein, fat, sugar and vitamins.
3. The system compares the day's totals with the user's calorie, protein, carbohydrate and fat goals.
4. The agent asks Claude for a short summary of the day with suggestions.
5. The system shows the totals, the goals and the summary. For a week, it shows the week's totals.

Alternative/Exception Flows:

1a The day has no meals: the system shows “No meals planned for this day.”

2a Nutrition data is missing for an ingredient: the totals are marked “incomplete”.

4a Claude is unavailable: the system shows a simple summary made from a template.

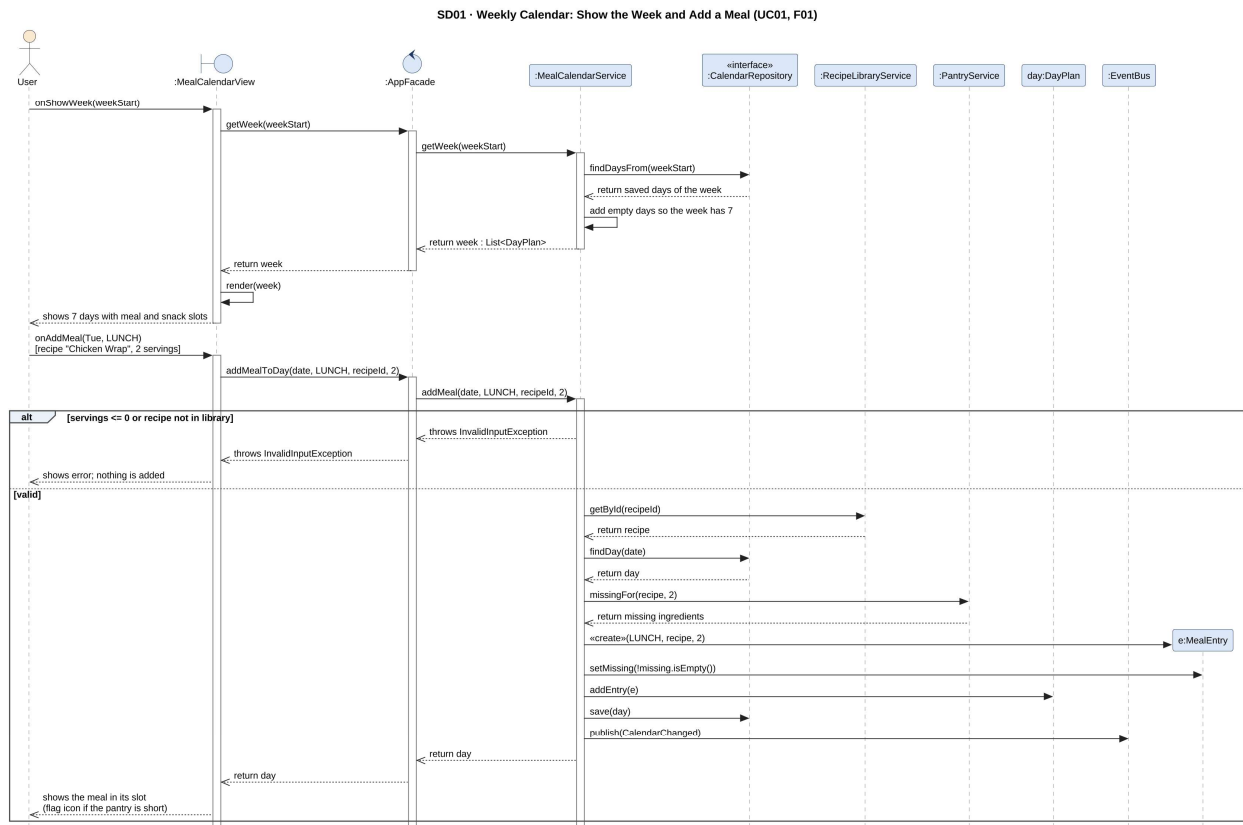
5a The user edits a goal: invalid values (for example 0 or negative) are rejected with a message; valid goals are saved and the summary is updated.

Postconditions: The summary is shown. Edited goals are saved in the profile.

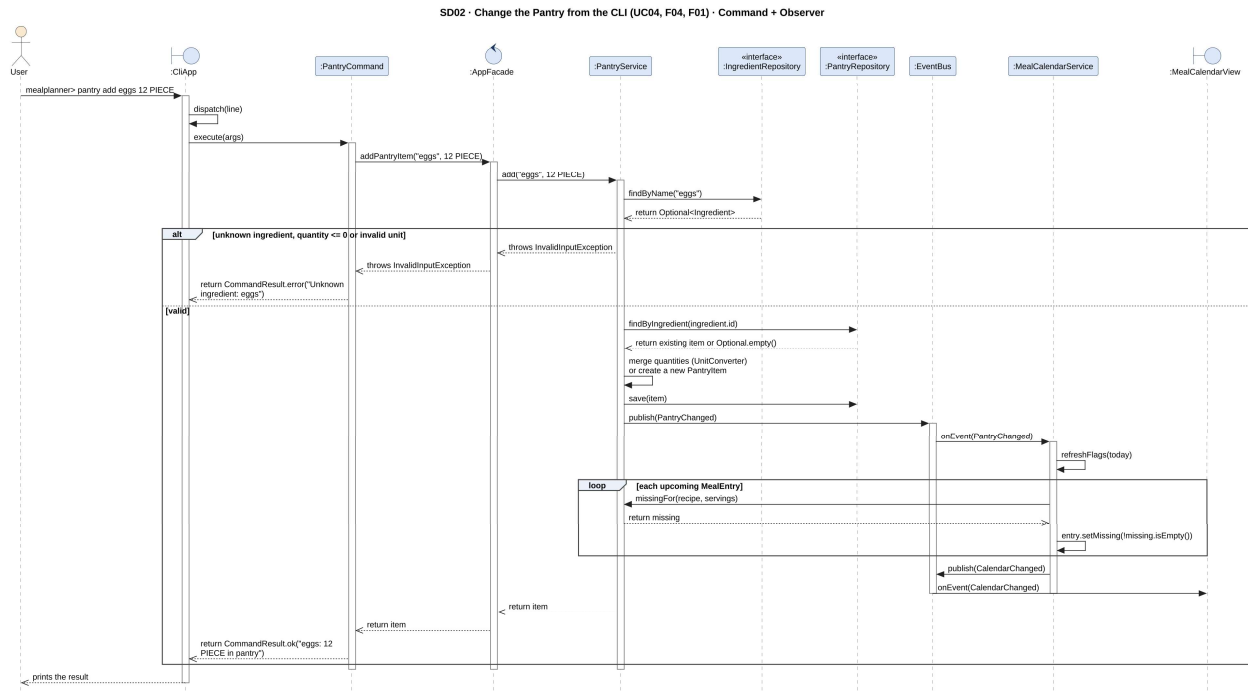
Related Feature(s): F10 Nutrition Summary (also uses F05 Macro Generator and F01 Weekly Meal Calendar)

2.3: Sequence Diagram

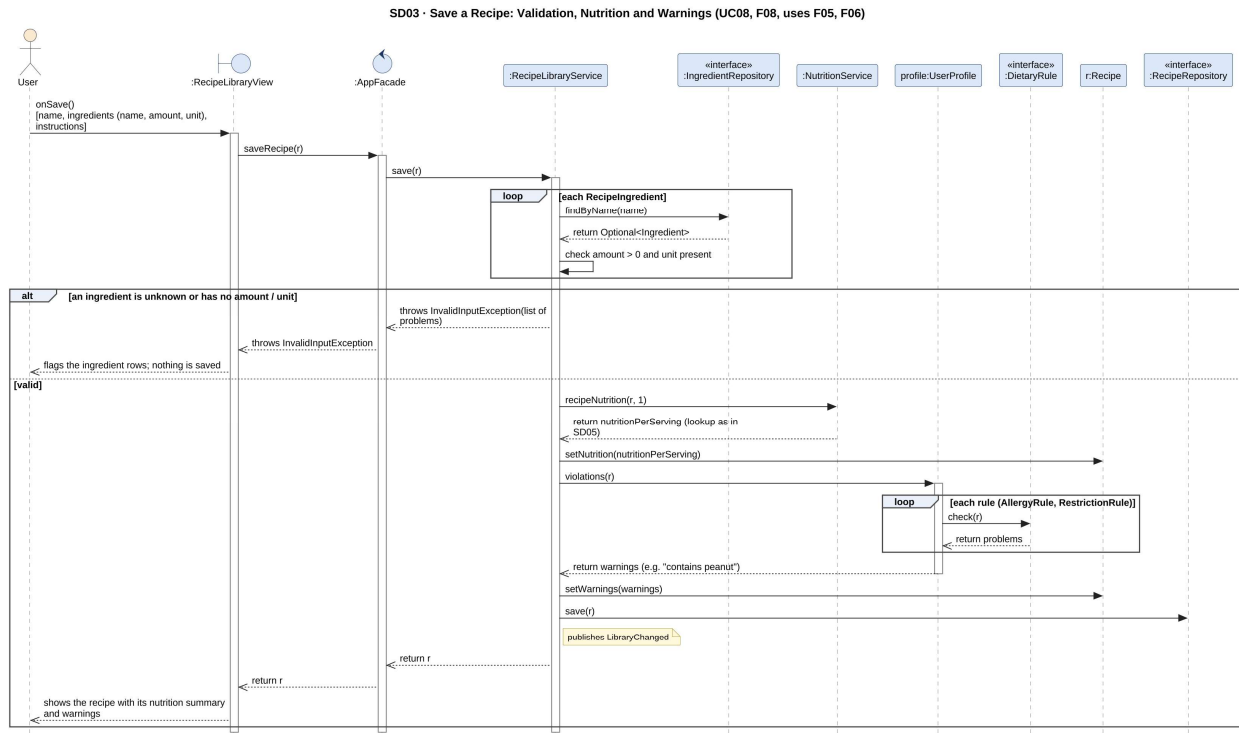
SD1: Add a Meal and Get the Daily Summary (UC01, F01)



SD2: Change the Pantry from the CLI (UC04, F04 (same structure: UC02, UC06, UC09))

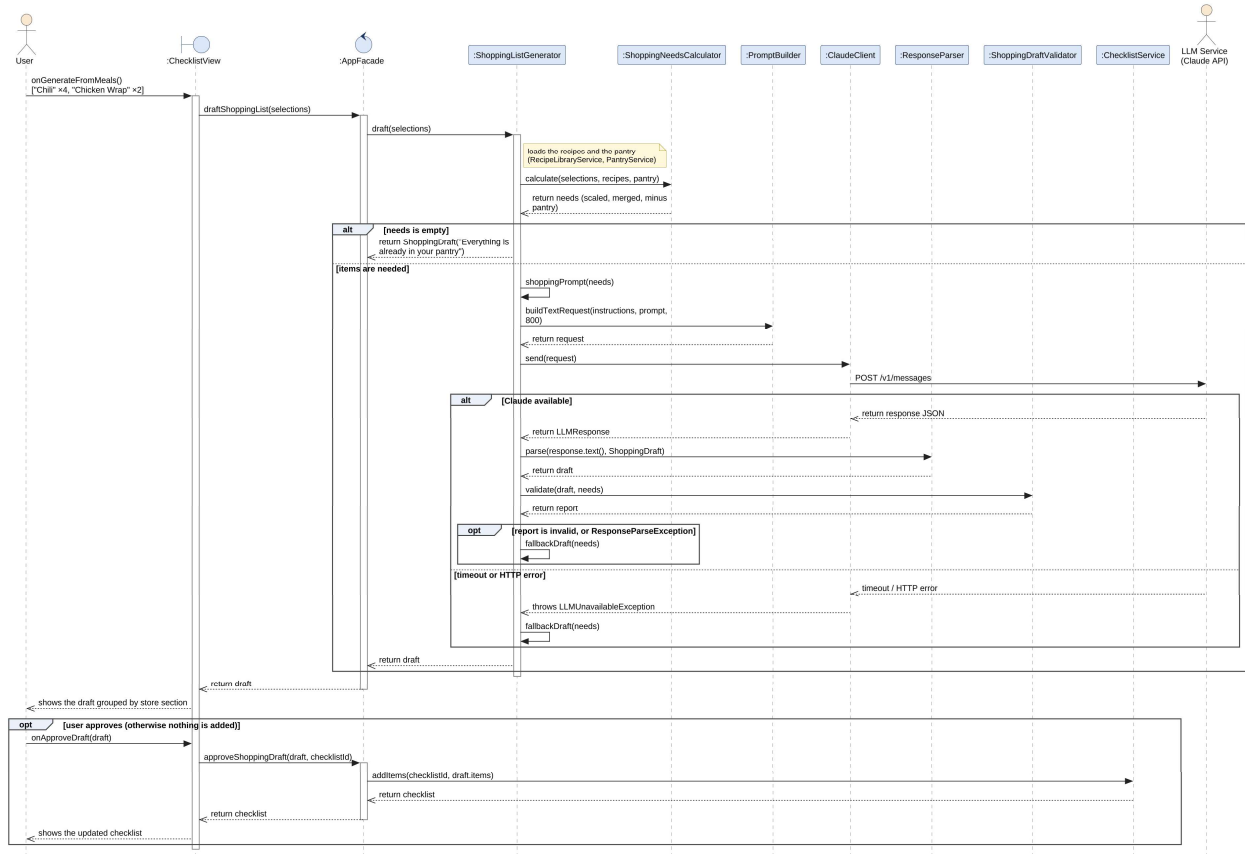


SD3: Save a Recipe (UC08, F08, uses F05, F06)



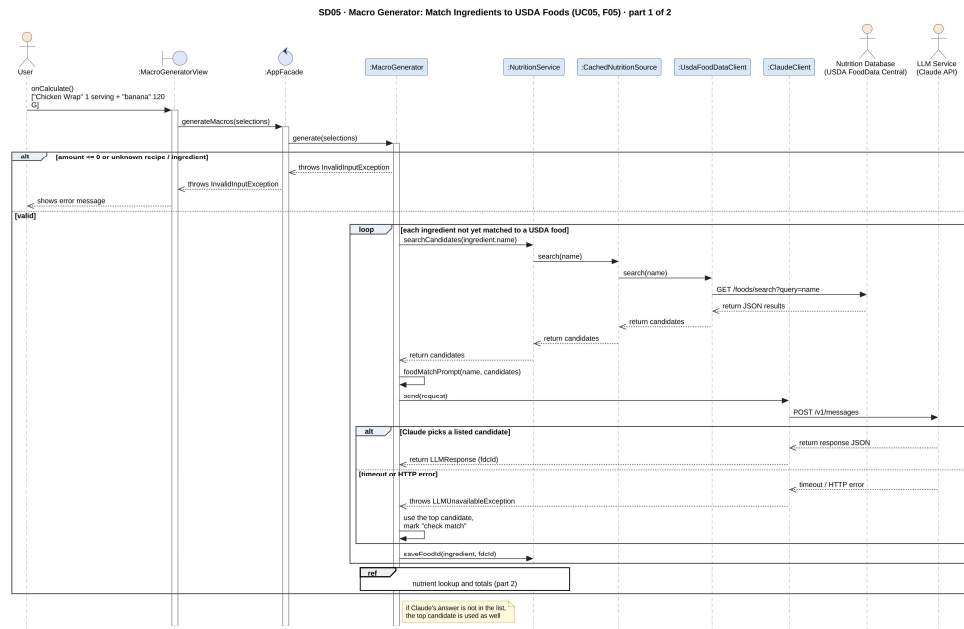
SD4: Generate a Shopping List (UC03, F03 → UC02, F02)

SD04 · Generate a Shopping List and Add It to a Checklist (UC03, F03 -- UC02, F02)

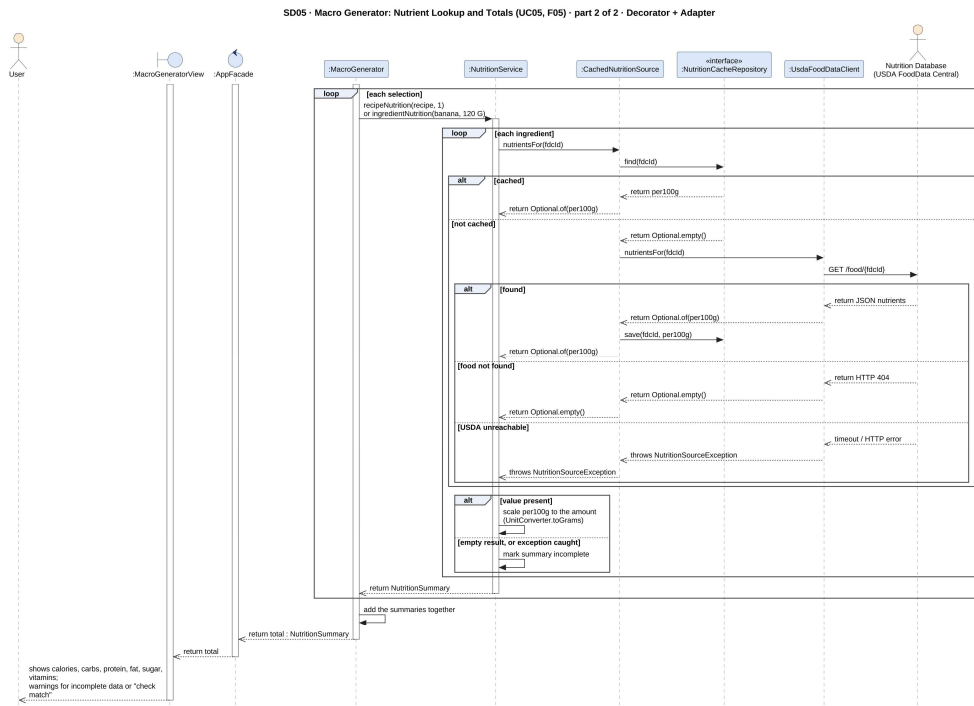


SD5: Macro Generator (UC05, F05)

Part 1:

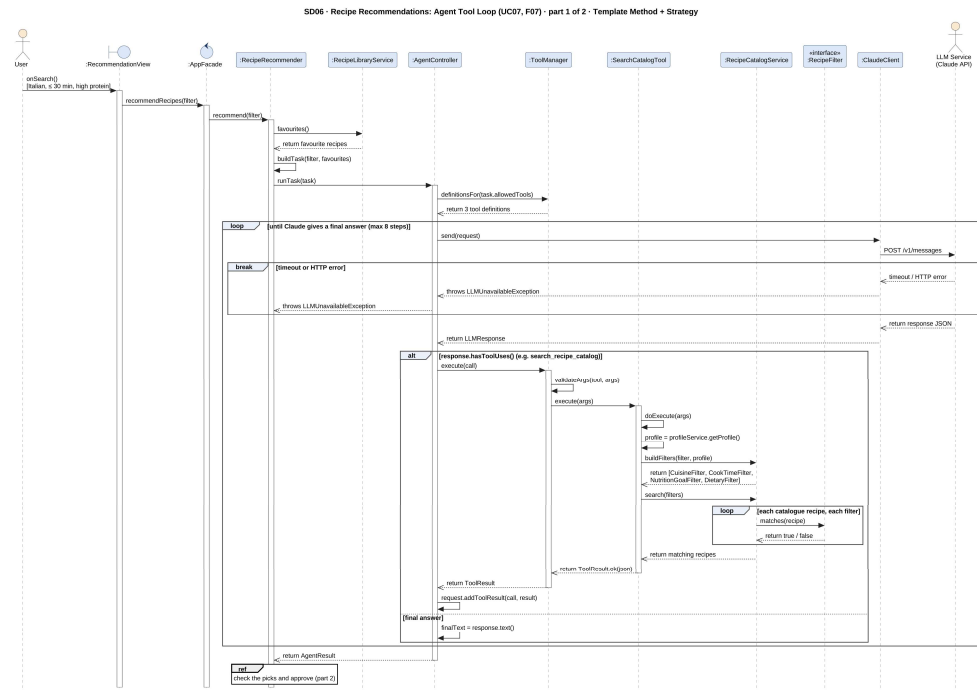


Part 2:

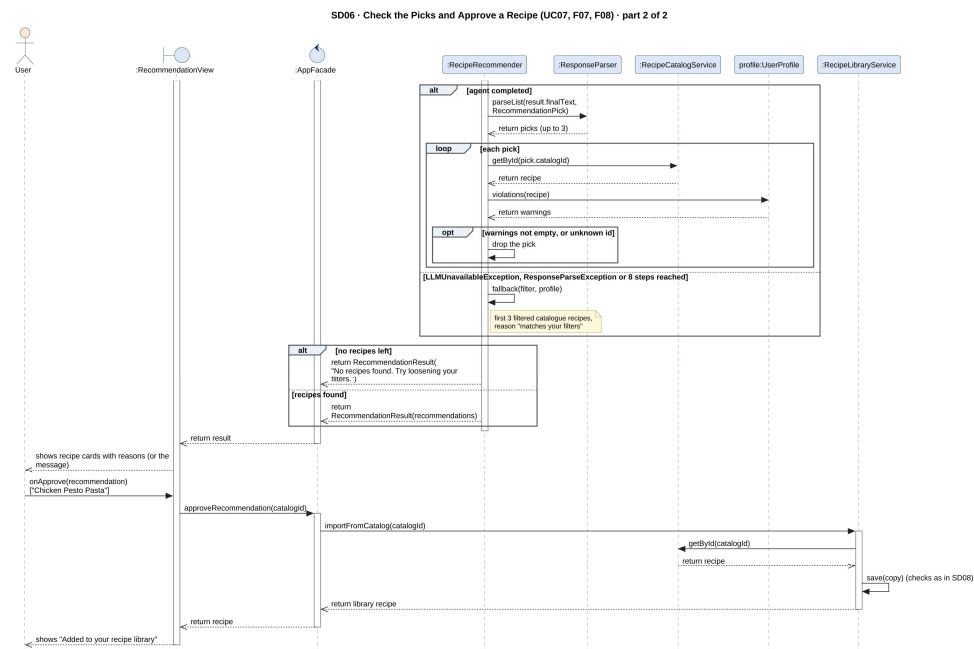


SD6: Recommend Recipes (UC10, F10 and UC07, F07)

Part 1:

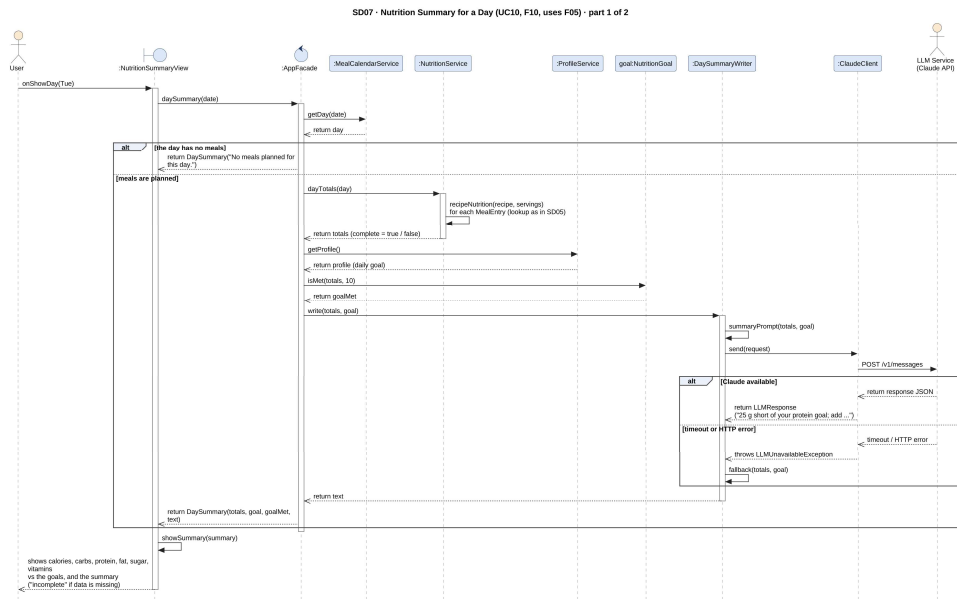


Part 2:

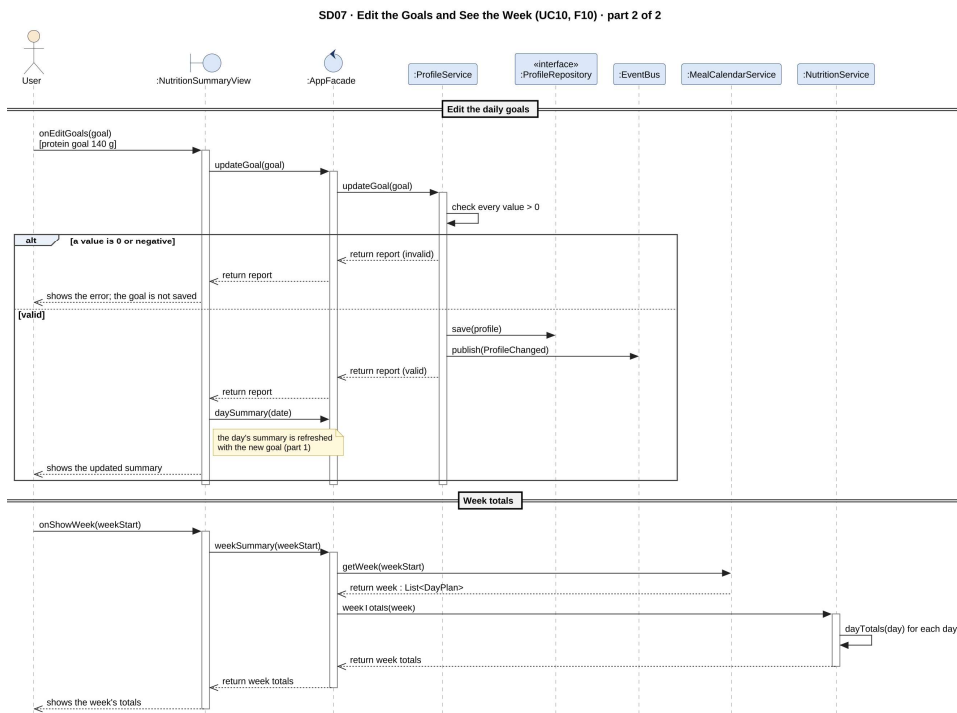


SD7: Nutrition Summary (UC10, F10, uses F05)

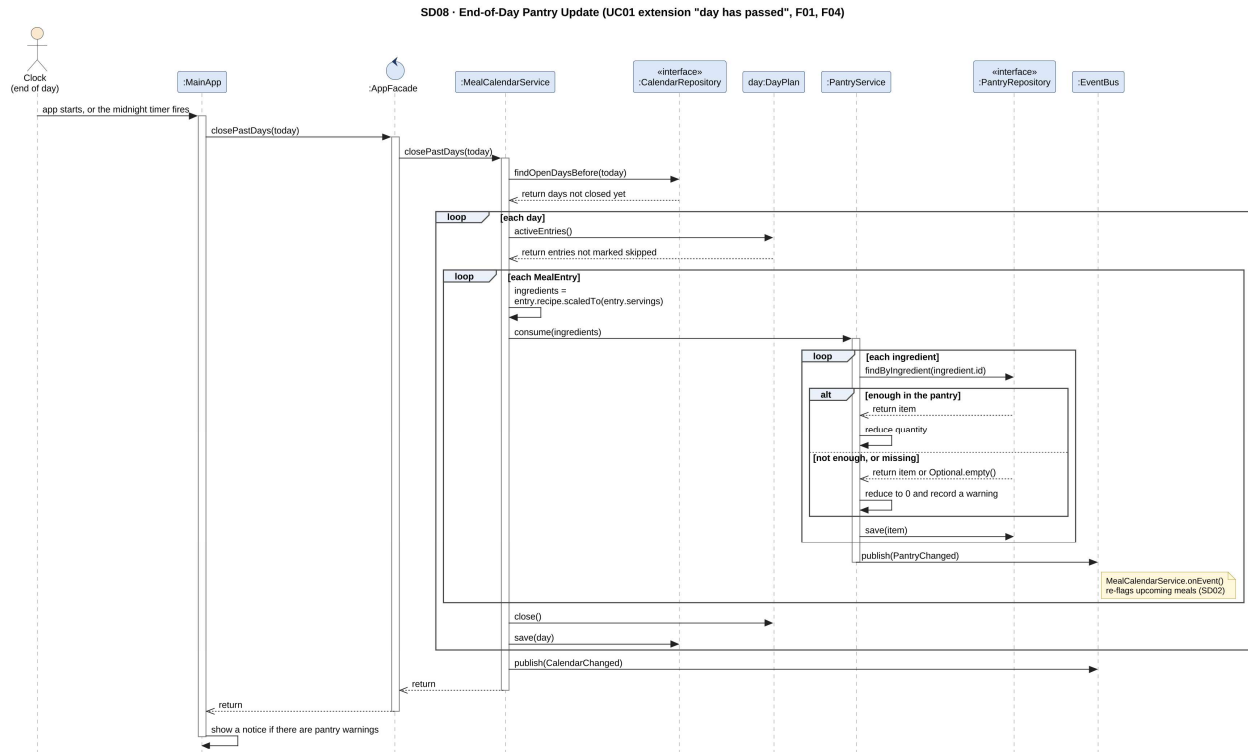
Part 1:



Part 2:



SD8: End-of-Day Pantry Update (UC01 extension, F01, F04)



Section 3: Feature-to-Design Traceability

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Calendar with meal and snack slots, flag meals the pantry cannot cover	Deterministic	UC01 Manage Meal Calendar and Track Macros	MealCalendarView, AppFacade, MealCalendarService, MealEntry, PantryService, NutritionService, DaySummaryWriter, ClaudeClient	onShowWeek(), getWeek(), onAddMeal(), addMealToDay(), addMeal(), missingFor(), closePastDays(), consume()	SD01, SD07	Facade, Observer
F02	Several checklists with items and quantities tick items off. Finish Shopping moves ticked items into the pantry and resets the list.	Deterministic	UC02 Use Shopping Checklist	ChecklistView, AppFacade, ChecklistService, Checklist, ChecklistItem, PantryService, UnitConverter, EventBus	createChecklist(), addChecklistItem(), setItemChecked(), finishShopping(), addPurchased(), removeChecked(), publish()	SD04; SD02 (Finish Shopping has the same structure: finishShopping() → addPurchased() → PantryChanged)	Facade, Observer
F03	Builds a shopping list from chosen recipes	Hybrid	UC03 Generate Shopping List	ChecklistView, AppFacade, ShoppingListGenerator, ShoppingNeedsCalculator, PromptBuilder, ClaudeClient, ResponseParser, ShoppingDraftValidator, ChecklistService	onGenerateFromMeals(), draftShoppingList(), draft(), calculate(), send(), validate(), fallbackDraft(), approveShoppingDraft(), addItems()	SD04	Facade, Adapter
F04	A space where user can list ingredients at home, a pantry inventory	Deterministic	UC04 Manage Pantry Inventory	PantryView, CliApp, PantryCommand, AppFacade, PantryService, IngredientRepository, PantryRepository, UnitConverter, EventBus	onAdd(), execute(), addPantryItem(), add(), findByName(), convert(), save(), publish()	SD02 (also SD07)	Command, Facade, Observer
F05	Nutrition summary (calories, carbohydrates, protein, fat, sugar, vitamins) for chosen recipes or pantry ingredients and amounts, from a reliable source.	Hybrid	UC05 Generate Macro Summary	MacroGeneratorView, AppFacade, MacroGenerator, NutritionService, ClaudeClient, CachedNutritionSource, NutritionCacheRepository, UsdaFoodDataClient	onCalculate(), generateMacros(), generate(), searchCandidates(), saveFoodId(), recipeNutrition(), ingredientNutrition(), nutrientsFor()	SD05 (parts 1–2)	Adapter, Decorator, Facade

F06	A profile where a user can set: dietary restrictions, allergies and favourite cuisines	Deterministic	UC06 Configure Profile	ProfileView, AppFacade, ProfileService, UserProfile, DietaryRule, AllergyRule, RestrictionRule, EventBus, RecipeLibraryService	onSave(), updateProfile(), update(), violations(), check(), publish(), refreshWarnings()	SD03 (dietary rules); SD02 (same save-and-event structure)	Observer, Facade,
F07	Recommend recipe for the user based on filters	Hybrid	UC07 Get Recipe Recommendations	RecommendationView, AppFacade, RecipeRecommender, AgentController, ToolManager, SearchCatalogTool, RecipeCatalogService, RecipeFilter, ClaudeClient, RecipeLibraryService	onSearch(), recommendRecipes(), recommend(), runTask(), execute(), search(), matches(), fallback(), approveRecommendation(), importFromCatalog()	SD06 (part 1-2)	Adapter, Facade
F08	Recipe Library where a user can add, edit or remove recipes with ingredients (amount and unit), cooking instructions, allergy and diet warnings and a nutrition summary.	Deterministic	UC08 Manage Recipe Library	RecipeLibraryView, AppFacade, RecipeLibraryService, IngredientRepository, NutritionService, UserProfile, Recipe, RecipeRepository	onSave(), saveRecipe(), save(), findByName(), recipeNutrition(), violations(), setWarnings(), removeRecipe(), archive()	SD03	Facade, Strategy
F09	The user can put a recipe under a 'favourite' filter. Favourites filter and rank higher in recommendations and meal plans.	Deterministic	UC09 Mark Favourite Recipe	RecipeLibraryView, AppFacade, RecipeLibraryService, Recipe, RecipeRepository, RecipeRecommender, MealPlanner	onToggleFavourite(), toggleFavourite(), setFavourite(), save(), favourites()	SD02 (same structure: toggleFavourite() → save()); SD06 (favourites in search results)	Facade
F10	User's nutrition for a day (calories, carbohydrates, protein, fat, sugar, vitamins) compared with editable daily goals	Hybrid	UC10 View Nutrition Summary	NutritionSummaryView, AppFacade, MealCalendarService, NutritionService, ProfileService, NutritionGoal, DaySummaryWriter, ClaudeClient	onShowDay(), daySummary(), dayTotals(), isMet(), write(), send(), updateGoal(), weekSummary(), weekTotals()	SD07 (parts 1–2)	Adapter, Facade

Section 4: How Each Feature are Realized

F01: Weekly Meal Calendar

Related Use Case: UC01 — Manage Weekly Meal Calendar

Related Sequence Diagrams: SD01 — Weekly Calendar: Show the Week and Add a Meal; SD08 — End-of-Day Pantry Update

Classes involved:

- **MealCalendarView** — shows the week with meal and snack slots for each day, and collects the recipe and servings.
- **AppFacade** — single entry point; passes the request to the right service.
- **MealCalendarService** — returns a week of days, adds meals, flags them, and closes past days.
- **CalendarRepository** — stores the days of the calendar.
- **DayPlan** — one day of the calendar with its meals.
- **MealEntry** — one meal in a slot, with its servings and a “missing ingredients” flag.
- **PantryService** — checks whether the pantry has enough and removes used ingredients.
- **EventBus** — publishes CalendarChanged so the screens refresh (Observer).

Important methods:

- | | |
|---------------------------------|---------------------------------------|
| • MealCalendarView.onShowWeek() | • MealCalendarService.addMeal() |
| • AppFacade.getWeek() | • PantryService.missingFor() |
| • MealCalendarService.getWeek() | • MealCalendarService.closePastDays() |
| • MealCalendarView.onAddMeal() | • PantryService.consume() |
| • AppFacade.addMealToDay() | |

Execution: When the user opens the calendar, MealCalendarView.onShowWeek() calls AppFacade.getWeek(), and MealCalendarService.getWeek() returns seven DayPlan objects, which the view shows as meal and snack slots. When the user picks a slot and a recipe, MealCalendarView.onAddMeal() calls AppFacade.addMealToDay(), which passes the request to MealCalendarService.addMeal(). The service checks the servings, asks PantryService.missingFor() whether the pantry has enough, creates a MealEntry (flagged if ingredients are missing), saves the day in the CalendarRepository and publishes CalendarChanged through the EventBus. When the app starts or at midnight, MealCalendarService.closePastDays() calls PantryService.consume() to remove the ingredients of every meal that was not skipped.

F02: Shopping Checklist

Related Use Case: UC02 — Use Shopping Checklist

Related Sequence Diagrams: SD04 — Generate a Shopping List (items added to a checklist); SD02 — Change the Pantry from the CLI (same structure as Finish Shopping)

Classes involved:

- **ChecklistView** — shows the checklists, their items and tick boxes, and the Finish Shopping button.
- **AppFacade** — entry point for checklist actions.
- **ChecklistService** — creates checklists, adds and ticks items, and runs Finish Shopping.
- **Checklist** — a named list of items; knows which items are ticked.
- **ChecklistItem** — one grocery item with a quantity and an “already in pantry” flag.
- **PantryService** — checks the pantry and adds the bought items.
- **UnitConverter** — combines quantities that use different units.
- **EventBus** — tells the rest of the app that the pantry changed (Observer).

Important methods:

- | | |
|------------------------------------|-------------------------------------|
| • AppFacade.createChecklist() | • ChecklistService.finishShopping() |
| • AppFacade.addChecklistItem() | • PantryService.addPurchased() |
| • ChecklistService.addItem() | • Checklist.removeChecked() |
| • PantryService.findByIngredient() | • EventBus.publish() |
| • AppFacade.setItemChecked() | |

Execution: The user creates a checklist with `AppFacade.createChecklist()` and adds items with `AppFacade.addChecklistItem()`. `ChecklistService.addItem()` calls `PantryService.findByIngredient()` and flags the item if it is already in the pantry. Ticking an item calls `AppFacade.setItemChecked()`. When the user clicks Finish Shopping, `ChecklistService.finishShopping()` passes the ticked items to `PantryService.addPurchased()`, which merges each one into the pantry using `UnitConverter`; unknown names are listed as “not added”. `Checklist.removeChecked()` then resets the list, and `EventBus.publish()` sends `PantryChanged` so the calendar flags update.

F03: Shopping List Generation

Related Use Case: UC03 — Generate Shopping List

Related Sequence Diagram: SD04 — Generate a Shopping List

Classes involved:

- **ChecklistView** — lets the user pick recipes and servings, shows the draft, and approves it.
- **AppFacade** — entry point.
- **ShoppingListGenerator** — runs the AI workflow for this feature.
- **ShoppingNeedsCalculator** — calculates the exact quantities: scales the recipes, merges them and subtracts the pantry.
- **PromptBuilder** — builds the request sent to Claude.
- **ClaudeClient** — sends the request to the Claude API (Adapter).
- **ResponseParser** — turns Claude's JSON reply into a `ShoppingDraft`.
- **ShoppingDraftValidator** — checks that Claude did not add, lose or lower any item.
- **ChecklistService** — adds the approved items to a checklist.

Important methods:

- | | |
|--|--|
| • <code>ChecklistView.onGenerateFromMeals()</code> | • <code>ResponseParser.parse()</code> |
| • <code>AppFacade.draftShoppingList()</code> | • <code>ShoppingDraftValidator.validate()</code> |
| • <code>ShoppingListGenerator.draft()</code> | • <code>ShoppingListGenerator.fallbackDraft()</code> |
| • <code>ShoppingNeedsCalculator.calculate()</code> | • <code>AppFacade.approveShoppingDraft()</code> |
| • <code>PromptBuilder.buildTextRequest()</code> | • <code>ChecklistService.addItem()</code> |
| • <code>ClaudeClient.send()</code> | |

Execution: When the user picks recipes and clicks Generate from meals, `ChecklistView.onGenerateFromMeals()` calls `AppFacade.draftShoppingList()`, which starts `ShoppingListGenerator.draft()`. The generator loads the recipes and the pantry, and `ShoppingNeedsCalculator.calculate()` works out exactly what is missing. It builds one request with `PromptBuilder.buildTextRequest()` and sends it with `ClaudeClient.send()`; Claude merges duplicate names and groups the items by store section. `ResponseParser.parse()` turns the reply into a `ShoppingDraft` and `ShoppingDraftValidator.validate()` checks it. If Claude fails or the check fails, `fallbackDraft()` gives a plain list. After the user approves, `AppFacade.approveShoppingDraft()` adds the items with `ChecklistService.addItem()`.

F04: Pantry Inventory

Related Use Case: UC04 — Manage Pantry Inventory

Related Sequence Diagrams: SD02 — Change the Pantry from the CLI; SD08 — End-of-Day Pantry Update (the pantry is also updated at the end of the day)

Classes involved:

- **PantryView** — GUI list of ingredients with add, edit and remove.
- **CliApp, PantryCommand** — the CLI version, for example `pantry add eggs 12 PIECE` (Command).
- **AppFacade** — entry point shared by the GUI and the CLI.
- **PantryService** — validates and saves pantry changes.
- **IngredientRepository** — the list of known ingredients, used to reject unknown names.
- **PantryRepository** — stores the pantry items.
- **UnitConverter** — merges quantities that use different units.
- **EventBus** — publishes `PantryChanged` (Observer).

Important methods:

- `PantryView.onAdd()`
- `PantryCommand.execute()`
- `AppFacade.addPantryItem()`
- `PantryService.add()`
- `IngredientRepository.findByName()`
- `UnitConverter.convert()`
- `PantryRepository.save()`
- `EventBus.publish()`

Execution: In the GUI, `PantryView.onAdd()` calls `AppFacade.addPantryItem()`; in the CLI, `CliApp` sends the typed line to `PantryCommand.execute()`, which calls the same method. `PantryService.add()` looks the name up with `IngredientRepository.findByName()` and checks the quantity and unit; unknown names and invalid values are rejected with a message. If the ingredient is already in the pantry, the quantities are merged with `UnitConverter.convert()`, and the item is saved with `PantryRepository.save()`. `EventBus.publish()` then sends `PantryChanged`, and `MealCalendarService` re-checks the flags on upcoming meals.

F05: Macro Generator

Related Use Case: UC05 — Generate Macro Summary

Related Sequence Diagram: SD05 — Macro Generator (parts 1–2)

Classes involved:

- **MacroGeneratorView** — lets the user choose recipes or pantry ingredients and amounts.
- **AppFacade** — entry point.
- **MacroGenerator** — runs the AI workflow: matches foods and adds up the totals.
- **NutritionService** — finds USDA foods and calculates the nutrients for an amount.
- **ClaudeClient** — asks Claude to pick the right USDA food (Adapter).
- **CachedNutritionSource** — returns saved nutrient values, or asks USDA (Decorator).
- **NutritionCacheRepository** — stores the nutrient values already fetched.
- **UsdaFoodDataClient** — talks to the USDA FoodData Central API (Adapter).

Important methods:

- `MacroGeneratorView.onCalculate()`
- `AppFacade.generateMacros()`
- `MacroGenerator.generate()`
- `NutritionService.searchCandidates()`
- `ClaudeClient.send()`
- `NutritionService.saveFoodId()`
- `NutritionService.recipeNutrition()`
- `NutritionService.ingredientNutrition()`
- `CachedNutritionSource.nutrientsFor()`
- `UsdaFoodDataClient.nutrientsFor()`

Execution: When the user clicks Calculate, `MacroGeneratorView.onCalculate()` calls `AppFacade.generateMacros()`, which starts `MacroGenerator.generate()`. Invalid amounts or unknown items are rejected. For each ingredient that is not yet linked to a USDA food, `NutritionService.searchCandidates()` gets a short list of matches and Claude, through `ClaudeClient.send()`, picks the right one; `NutritionService.saveFoodId()` stores it so Claude is asked only once. `NutritionService.recipeNutrition()` and `ingredientNutrition()` then get the values per 100 g from `CachedNutritionSource.nutrientsFor()`, which uses the cache or `UsdaFoodDataClient`, and scale them to the amounts. `MacroGenerator` adds everything up; a missing food or an unreachable USDA service marks the summary “incomplete”.

F06: Profile Configuration

Related Use Case: UC06 — Configure Profile

Related Sequence Diagrams: SD03 — Save a Recipe (the dietary rules); SD02 — Change the Pantry from the CLI (same save-and-event structure)

Classes involved:

- **ProfileView** — form for dietary restrictions, allergies and favourite cuisines.
- **AppFacade** — entry point.
- **ProfileService** — validates and saves the profile.
- **UserProfile** — holds the settings and creates the dietary rules.
- **DietaryRule, AllergyRule, RestrictionRule** — one rule each; checks a recipe (Strategy).
- **EventBus** — publishes `ProfileChanged` (Observer).
- **RecipeLibraryService** — refreshes the recipe warnings when the profile changes.

Important methods:

- ProfileView onSave()
- AppFacade.updateProfile()
- ProfileService.update()
- EventBus.publish()
- RecipeLibraryService.refreshWarnings()
- UserProfile.violations()
- DietaryRule.check()

Execution: When the user clicks Save, ProfileView.onSave() calls AppFacade.updateProfile(), which calls ProfileService.update(). The service checks each value (allergy names must be known ingredients); if something is wrong it returns a report and saves nothing. Otherwise it saves the profile and EventBus.publish() sends ProfileChanged. RecipeLibraryService.refreshWarnings() then re-checks every recipe with UserProfile.violations(), which runs each DietaryRule.check(). The same rules are strict filters in F07 and F08, while favourite cuisines are only preferences.

F07: Recipe Recommendations

Related Use Case: UC07 — Get Recipe Recommendations

Related Sequence Diagram: SD06 — Recipe Recommendations (AI Agent) (parts 1–2)

Classes involved:

- **RecommendationView** — filter form (cuisine, cooking time, nutrition goal) and recipe cards.
- **AppFacade** — entry point.
- **RecipeRecommender** — runs the AI workflow for this feature.
- **AgentController** — runs the tool loop with Claude (max 8 steps).
- **ToolManager** — gives Claude the definitions of the three tools and runs the tools it asks for.
- **SearchCatalogTool** — the search_recipe_catalog tool (Template Method).
- **RecipeCatalogService** — searches the built-in recipe catalogue.
- **RecipeFilter** — cuisine, cooking-time, nutrition-goal and dietary filters (Strategy).
- **ClaudeClient** — sends each request to the Claude API (Adapter).
- **RecipeLibraryService** — gives the favourites and imports an approved recipe.

Important methods:

- RecommendationView.onSearch()
- AppFacade.recommendRecipes()
- RecipeRecommender.recommend()
- AgentController.runTask()
- ToolManager.execute()
- RecipeCatalogService.search()
- RecipeFilter.matches()
- RecipeRecommender.fallback()
- AppFacade.approveRecommendation()
- RecipeLibraryService.importFromCatalog()

Execution: When the user clicks Search, RecommendationView.onSearch() calls AppFacade.recommendRecipes(), which starts RecipeRecommender.recommend(). The recommender adds the user's favourites and runs AgentController.runTask(). Claude can call three read-only tools; when it asks for search_recipe_catalog, ToolManager.execute() runs SearchCatalogTool, which calls RecipeCatalogService.search() with the RecipeFilter strategies (each one calls matches() on a recipe), and the result goes back to Claude. Claude returns up to 3 recipes with reasons, and any pick that breaks the profile's rules is dropped. If Claude is unavailable, RecipeRecommender.fallback() shows the first filtered recipes; if nothing is left, the user is asked to loosen the filters. When the user approves a card, AppFacade.approveRecommendation() calls RecipeLibraryService.importFromCatalog().

F08: Recipe Library

Related Use Case: UC08 — Manage Recipe Library

Related Sequence Diagram: SD03 — Save a Recipe

Classes involved:

- **RecipeLibraryView** — recipe form (ingredients, instructions) and the library list.
- **AppFacade** — entry point.
- **RecipeLibraryService** — validates, completes and saves recipes.
- **IngredientRepository** — used to reject unknown ingredient names.
- **NutritionService** — calculates the nutrition per serving.
- **UserProfile** — creates the allergy and diet warnings.
- **Recipe** — the recipe with its ingredients, warnings, nutrition and archived flag.
- **RecipeRepository** — stores the recipes.
- **EventBus** — sends LibraryChanged so the library screen refreshes.

Important methods:

- | | |
|--------------------------------------|----------------------------|
| • RecipeLibraryView onSave() | • Recipe.setWarnings() |
| • AppFacade.saveRecipe() | • RecipeRepository.save() |
| • RecipeLibraryService.save() | • EventBus.publish() |
| • IngredientRepository.findByName() | • AppFacade.removeRecipe() |
| • NutritionService.recipeNutrition() | • Recipe.archive() |
| • UserProfile.violations() | |

Execution: When the user clicks Save, RecipeLibraryView.onSave() calls AppFacade.saveRecipe(), which calls RecipeLibraryService.save(). The service checks every ingredient with IngredientRepository.findByName() and makes sure it has an amount and a unit; problems are flagged and nothing is saved. It then gets the nutrition per serving from NutritionService.recipeNutrition(), gets the warnings from UserProfile.violations(), stores them with Recipe.setWarnings(), and saves the recipe with RecipeRepository.save(); EventBus.publish() then sends LibraryChanged so the library screen refreshes. Removing a recipe calls AppFacade.removeRecipe(), which archives it with Recipe.archive(), so meals already in the calendar still work.

F09: Favourite Recipe

Related Use Case: UC09 — Mark Favourite Recipe

Related Sequence Diagrams: SD02 — Change the Pantry from the CLI (same structure); SD06 — Recipe Recommendations (AI Agent) (favourites passed to the recommender)

Classes involved:

- **RecipeLibraryView** — shows the heart icon and the Favourites filter.
- **AppFacade** — entry point.
- **RecipeLibraryService** — switches and saves the favourite status and returns the favourites.
- **Recipe** — holds the favourite flag.
- **RecipeRepository** — stores the change.
- **RecipeRecommender** — uses the favourites to rank recipes higher.

Important methods:

- | | |
|--|-------------------------------------|
| • RecipeLibraryView.onToggleFavourite() | • Recipe.setFavourite() |
| • AppFacade.toggleFavourite() | • RecipeRepository.save() |
| • RecipeLibraryService.toggleFavourite() | • RecipeLibraryService.favourites() |

Execution: When the user clicks the heart, RecipeLibraryView.onToggleFavourite() calls AppFacade.toggleFavourite(), which calls RecipeLibraryService.toggleFavourite(). The service loads the recipe, switches it with Recipe.setFavourite() and saves it with RecipeRepository.save(); if saving fails, the heart goes back and an error is shown. The Favourites filter shows the recipes returned by RecipeLibraryService.favourites(), and RecipeRecommender passes the same list to Claude so favourites rank higher.

F10: Nutrition Summary

Related Use Case: UC10 — View Nutrition Summary

Related Sequence Diagram: SD07 — Nutrition Summary (parts 1–2)

Classes involved:

- **NutritionSummaryView** — shows the day's or week's nutrition and the goals, and lets the user edit the goals.
- **AppFacade** — entry point.
- **MealCalendarService** — gives the meals planned for the day or week.
- **NutritionService** — adds up the nutrients of the day's or week's meals (lookup through the cache and USDA).
- **ProfileService** — gives and saves the daily goals.
- **NutritionGoal** — the calorie, protein, carbohydrate and fat goals, checks whether the totals meet them.
- **DaySummaryWriter** — asks Claude for a short summary of the day, with a template fallback.
- **ClaudeClient** — sends the request to the Claude API (Adapter).

Important methods:

- | | |
|------------------------------------|---------------------------------|
| • NutritionSummaryView.onShowDay() | • ClaudeClient.send() |
| • AppFacade.daySummary() | • AppFacade.updateGoal() |
| • MealCalendarService.getDay() | • ProfileService.updateGoal() |
| • NutritionService.dayTotals() | • AppFacade.weekSummary() |
| • NutritionGoal.isMet() | • NutritionService.weekTotals() |
| • DaySummaryWriter.write() | |

Execution: When the user opens a day, NutritionSummaryView.onShowDay() calls AppFacade.daySummary(). The facade gets the day's meals with MealCalendarService.getDay(); if there are none, the user sees “No meals planned for this day.” Otherwise NutritionService.dayTotals() adds up the calories, carbohydrates, protein, fat, sugar and vitamins, NutritionGoal.isMet() compares them with the daily goal, and DaySummaryWriter.write() sends the totals and goal to Claude through ClaudeClient.send() for a short summary with suggestions. If Claude is unavailable, a template summary is used, and missing nutrition data marks the totals “incomplete”. When the user edits a goal, AppFacade.updateGoal() calls ProfileService.updateGoal(), which rejects values of 0 or less and saves valid goals. For the week view, AppFacade.weekSummary() adds up the seven days with NutritionService.weekTotals().